

ONLINE AUCTION & BIDDING PLATFORM

WITH REAL-TIME CONCURRENCY HANDLING

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the course

DATABASE SYSTEMS AND DATABASE ENGINEERING (DSDBE)

BACHELOR OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

K L (Deemed to be) UNIVERSITY

Team 22 — Section 7

Param Manasvi-2520030072

Ch. Shravya Chowdary-2520030322

Academic Year 2026–27

ACKNOWLEDGEMENT

The successful completion of the Online Auction & Bidding Platform was made possible through the guidance, support and cooperation received during the development and evaluation of the project. We express our sincere gratitude to the faculty and department for providing the academic environment, resources and project-based learning framework required to design, implement and test the system.

We acknowledge the support received from classmates, friends and family during implementation, debugging, database testing and documentation. The project provided practical exposure to relational schema design, REST API development, authentication, transactions, concurrency control and real-time communication.

The work also benefited from the DSDBE course structure, particularly the topics of backend layering, ER modelling, SQL, transactions, indexing and distributed-database fundamentals. These topics were connected to a practical auction workload in which the correctness of a shared database state is central.

Team 22 — Section 7

Param Manasvi-2520030072

Ch. Shravya Chowdary-2520030322

ABSTRACT

Online auction systems require more than simple item listings and bid submission. They must maintain correct bid state when multiple users submit bids at nearly the same time, provide clear auction lifecycle management, authenticate users and expose the latest state to connected clients. This project presents Vault, an Online Auction & Bidding Platform that combines a React-based web interface, an Express.js REST backend and a MySQL 8.0 relational database using the InnoDB storage engine. The platform supports registration and login, auction browsing, search and filtering, watchlists, recommendations, live auctions, bid history, venue visits, directions, notifications and a separate administrator control center.

The central technical focus is real-time concurrency handling. Bid placement is implemented as a transaction that locks the target auction row using `SELECT ... FOR UPDATE`, validates the incoming bid against the locked current highest bid, updates the auction, inserts the bid history record and commits the transaction as one unit. Socket.IO broadcasts bid updates and outbid notifications to connected users. A background node-cron job activates scheduled auctions and closes expired auctions while recording the winner. The database schema uses primary keys, foreign keys, composite keys, constraints and an auction/amount index to support integrity and retrieval. The implementation therefore demonstrates the practical application of backend layering, relational design, SQL, transactions and concurrency control in an auction domain.

INDEX

S. No.	Chapter / Section	Contents	Page
1	Introduction	Background, problem statement, scope, objectives, importance, target users	5–15
2	System Requirements	Hardware, software, development requirements	16–21
3	Technology Stack	Frontend, backend, database, version control, APIs/external services	22–32
4	System Architecture	High-level architecture, layers, deployment architecture	33–36
5	Design	ER diagram / relational database schema	37
6	Implementation	Modules, frontend logic, API integration, authentication, concurrency	38–49
7	Features	User features and separate administrator features	50–57
8	Testing	Functional, integration, concurrency and database testing	58–66
9	Deployment	Database, backend and frontend setup	67–75
10	Challenges & Limitations	Development issues, solutions and known limitations	76–84
11	Future Enhancements	Payments, scaling, analytics and operational improvements	85–90
12	Conclusion	Summary, achievements and skills learned	91–99
13	References	Books, tutorials, APIs and documentation	100–101
14	Appendices	Screenshots, code snippets, setup and user guide	102–109

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Online auction systems combine product discovery, time-bound transactions and competitive bidding in one digital workflow. Unlike a conventional catalogue, an auction must preserve a single authoritative bid state while several users may submit requests close together. The project named Vault addresses this requirement through a web-based auction platform backed by MySQL and an Express server.

The application provides a public-facing marketplace where users can discover active and scheduled auctions, inspect product information, filter listings, place bids, receive live updates, maintain a watchlist and review activity. A separate administrator control center is protected by role-based authorization and provides operational management for products, auctions, approvals, users and bid records.

The database is not treated as a passive storage layer. The bid workflow uses a transaction and `SELECT ... FOR UPDATE` so that the auction row becomes the synchronization point for competing requests. This makes the database responsible for preserving the auction invariant: the accepted current bid must be based on the latest committed state.

Auctions are among the oldest price-discovery mechanisms in commerce. In a traditional auction hall the auctioneer is the single point of coordination: bids are heard in sequence, the highest offer is announced and the hammer falls at a definite moment. Moving the same process to the web removes the physical venue and the geographical limits on participation, but it also removes the auctioneer. The software must now play that role, deciding in a fraction of a second which of several near-simultaneous requests is the valid highest bid.

Vault is designed around that responsibility. Its client-server structure keeps presentation in the browser, business rules in the Express application layer and authoritative state in the MySQL database. Registered users browse listings that are scheduled or live, while administrators approve submissions, manage the catalogue and monitor activity from a separate interface. Every bid is accepted or rejected by the server and never by the browser, so a modified client cannot bypass the rules of an auction.

The platform also addresses the time dimension of auctions. Each auction has a start time and an end time, and the system must change its state at the correct moment without waiting for a person to intervene. A background job activates scheduled auctions, closes expired ones and records the winner, while Socket.IO pushes the resulting changes to every connected browser so that current bids and notifications stay in step with the database.

From an academic viewpoint the project was developed as an applied study for the Database Systems and Database Engineering course. It brings together ER modelling, SQL and constraints, transactions, row-level locking and indexing in one working application, and it keeps the discussion grounded in a workload where an incorrect result, such as two bidders each believing they hold the highest bid, is immediately visible and testable.

1.2 Problem Statement

The problem addressed

The main problem is maintaining correctness when multiple users bid for the same auction at nearly the same time. A simple read-check-write sequence can allow two requests to read the same current highest bid and then overwrite one another. This creates a lost-update condition and can produce an incorrect final state.

The project therefore needs an architecture that authenticates users, separates administrative operations, stores normalized relational data, validates bids on the server, protects the critical row with a transaction, and broadcasts the committed result to connected clients.

The difficulty can be seen with a small example. Suppose the current highest bid on an auction is 500 and two bidders submit 600 and 550 at almost the same instant. If both requests read the value 500, each concludes that its own amount is acceptable, and whichever write happens last becomes the stored highest bid. If the lower bid is written last, the higher bid of 600 disappears from the auction state even though it was acknowledged. Figure 1.1 contrasts this unsafe sequence with the transactional approach adopted in this project.

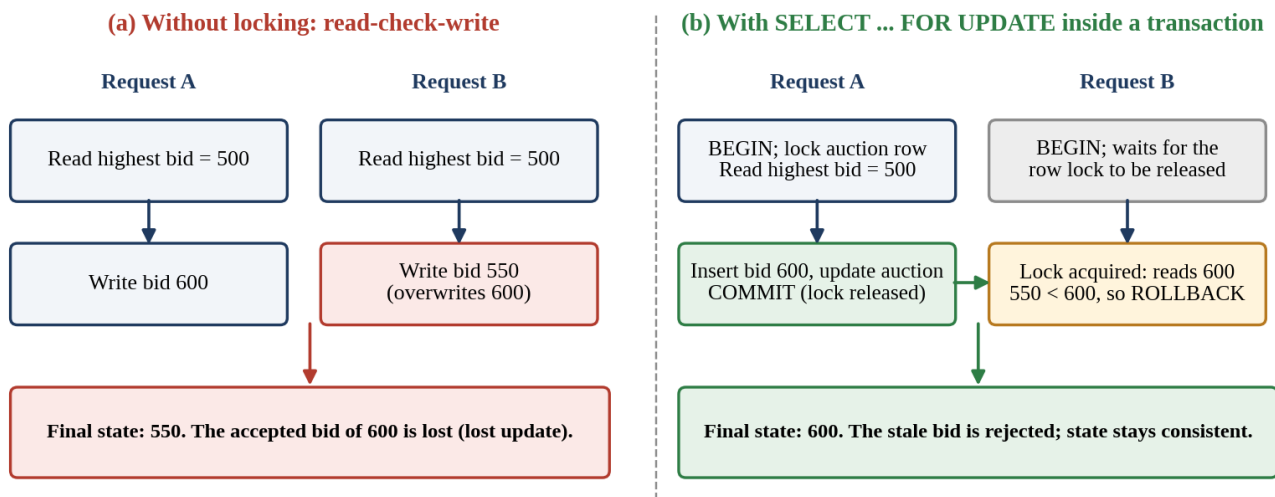


Figure 1.1 Lost-update race without locking compared with the row-locked bid transaction.

Concurrency is not the only concern. An auction platform must also decide who may perform which action, because bidding, approving listings and deleting users require different privileges. It must keep time-dependent state correct, since an auction that is not closed at its end time would continue to accept bids. It must keep connected clients informed, because a bidder looking at a stale price makes decisions on wrong information. Finally, it must preserve referential integrity so that bids, results, watchlists and visits never refer to auctions or users that no longer exist.

1.3 Scope of the Project

Functional scope

The implemented scope covers registration and login, JWT-based authentication, public auction browsing, search and filters, auction details, bid history, bidding, watchlists, recommendations, activity history, live-auction venue information, visit scheduling, directions, notifications and a floating assistant.

The administrative scope is intentionally separate. Administrators can view dashboard statistics, approve or reject auction submissions, create and manage products, create and manage auctions, inspect users, change user roles, inspect bids and remove accounts subject to database constraints.

Technical scope

The technical scope covers the complete path from the browser to the database: a React interface, REST endpoints in Express, JWT authentication with an administrator guard, a seven-table MySQL schema with primary keys, foreign keys, composite keys and an auction/amount index, a transactional bid procedure, Socket.IO rooms for live events and a node-cron job for lifecycle changes. A concurrency test script is included so that the correctness claim can be demonstrated rather than only described.

The scope also includes the full auction lifecycle shown in Figure 1.2. A submitted auction first waits for administrator approval; an approved auction is scheduled, becomes active when its start time arrives and is closed when its end time passes, at which point the winning bid is stored in the `auction_results` table.

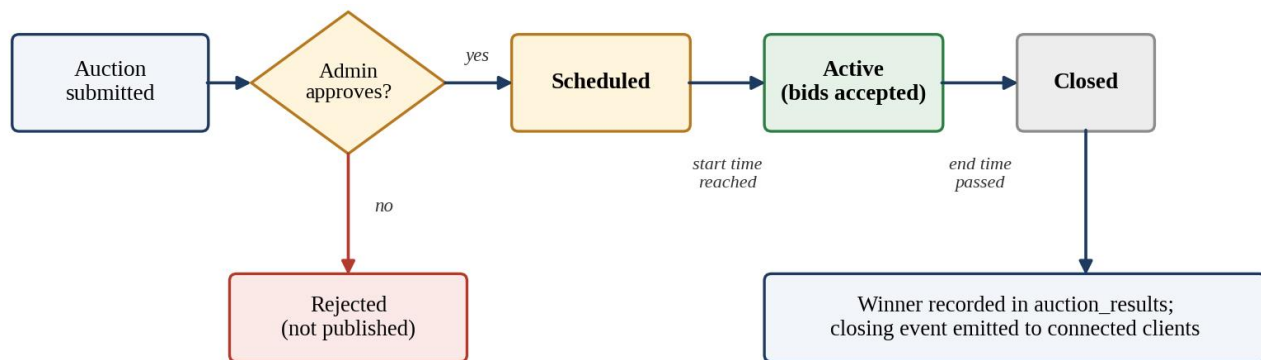


Figure 1.2 Auction lifecycle from submission to recorded result.

Boundaries of the project

Online payment processing, large-scale horizontal scaling and advanced analytics are treated as future enhancements and are discussed in Chapter 11. The present work concentrates on the correctness of shared auction state, which is the part of the problem most closely tied to database engineering.

1.4 Objectives

Project objectives

Design a relational database that separates users, products, auctions, bids and supporting entities while preserving referential integrity.

Implement a layered backend with routes, authentication middleware, administrator authorization, controllers and a MySQL connection pool.

Implement a concurrency-safe bid transaction using row-level locking and commit/rollback semantics.

Provide real-time bid and outbid notifications using Socket.IO and automate auction activation and closure using node-cron.

Provide a usable interface for both normal users and administrators, with clear separation between their responsibilities.

Each objective was paired with a concrete implementation artefact and a way of checking it, so that completion can be judged from evidence and not only from the description of intent. The table below summarises this mapping; the later chapters on implementation and testing give the details.

Objective	Realised through	Checked by
Relational design	Seven tables with primary, foreign and composite keys and constraints	schema.sql and the ER diagram (Chapter 5)
Layered backend	Routes, JWT and admin middleware, controllers, MySQL pool	Endpoint and role checks (Chapter 8)
Concurrency-safe bidding	Transaction with SELECT ... FOR UPDATE, commit and rollback	concurrentBidTest.js (Chapter 8)
Real-time and automation	Socket.IO rooms; node-cron job running every 30 seconds	Live update and auction closure behaviour
Dual interface	User pages and a separate Admin Control Center	Functional testing of both interfaces

Together these objectives ensure that the project is judged on the database behaviour that matters in an auction (integrity, isolation and timely state change) as well as on the completeness of the user interface.

Design principles

Three principles guided how the objectives were met. First, the server is the source of truth: the client may suggest an action, but only the server decides whether it is valid. Second, privileges are kept as narrow as possible, so ordinary accounts never reach administrative routes. Third, the critical section of the bid transaction is kept short, locking a single auction row rather than a whole table, so that unrelated auctions continue to be served while one is being updated.

1.5 Importance of the Application

Why the system is technically relevant

The application is useful as a database engineering case study because an auction creates a clear correctness invariant that can be tested. The highest bid is not merely a UI value; it is persistent state that must remain consistent across concurrent requests.

The project connects course concepts to implementation: relational modelling is represented by the seven-table schema, SQL is used by controllers, transactions and locks are used for bidding, composite indexes support retrieval, and role checks protect administrative operations.

Practical and academic value

In practical terms the system shows how a small web application can offer behaviour that users expect from commercial auction sites: live prices, outbid alerts, saved auctions, venue information for live events and a review step before listings become public. Each of these features depends on the database being treated as the source of truth rather than as a log of what the interface displayed.

The project is also valuable as a teaching artefact because its invariants can be stated precisely and tested. The current highest bid of an auction must never decrease; every accepted bid must have a corresponding row in the bid history; an auction must have at most one result row; and a closed auction must not accept further bids. Declaring these rules makes it possible to check them with queries and with the concurrency test script, which turns an abstract discussion of isolation into observable evidence.

Finally, the work is relevant to maintainability. The separation into routes, middleware, controllers, sockets and jobs lets a new developer find where each rule is enforced, and environment-based configuration allows the same code to run on different machines without editing source files.

The auction workload also provides a natural starting point for discussing distributed-database topics. If data were replicated or partitioned by auction, each auction would still need one authoritative copy of its highest bid, which shows why a distributed design must decide where the lock or the leader for a given row resides. Although Vault runs against a single MySQL instance, the same reasoning explains why bid correctness should not be delegated to individual application nodes or to browsers.

1.6 Target Users

Primary user groups

Normal users are buyers or bidders who register, browse listings, place bids, track auctions and use personal features such as watchlists and activity history.

Administrators are operational users with a separate protected control center. They manage catalogue data, approvals, auction lifecycle data, user roles and bid records.

System maintainers and developers form a third audience because the project includes a clear client/server structure, environment configuration, schema and a concurrency test script.

The needs of these groups differ, and the design reflects those differences. Bidders value speed, clarity and trust: they want to see the current price and an immediate response when a bid is accepted or rejected. Administrators value control and traceability: they need to see what is waiting for approval, who holds which role and which bids were placed. Maintainers value structure and reproducibility: they need predictable folders, schema files and test scripts. The table below summarises how access is divided.

User group	Access level	Typical actions
Visitor	Public pages, no login	Browse listings, view auction details and bid history
Registered user	JWT-authenticated session	Place bids, manage watchlist, schedule venue visits, receive notifications, review activity
Administrator	JWT with admin role, isAdmin guard	Use the Control Center: approvals, products, auctions, users and bids
Maintainer / developer	Source code and environment files	Configure environments, load schema and seed data, run the concurrency test

Design implications

Designing for several audiences influenced the feature set. Public pages are readable without an account so that visitors can judge the catalogue before registering; personal features such as watchlists, recommendations and activity history reward registration; and the administrator interface sits behind the role check so that operational power is never exposed to ordinary accounts. Maintainers benefit from the separation of client and server folders and from the schema and seed files, which allow the database to be rebuilt quickly on a new machine.

1.7 User Workflow

Normal user workflow

A typical user journey begins with registration or login. After authentication, the user can browse auctions and use search, category, status, price, featured and certification filters. Selecting an auction opens the detail view where the current bid and bid history can be reviewed.

The user can submit a bid manually or use quick bid actions. The server validates the auction status, end time and amount while holding the auction row lock. Once committed, the new state is broadcast to connected clients and the previous highest bidder can receive an outbid event.

Figure 1.3 summarises this journey. After login the server issues a JWT, which the client stores and attaches to each protected request through the Axios interceptor. The user narrows the catalogue with search and filters, opens an auction and reviews the current bid, the bid history and the remaining time. Optional actions such as adding the auction to the watchlist, scheduling a venue visit for a live auction or requesting directions are available from the detail and live-auction views without interrupting the bidding flow.

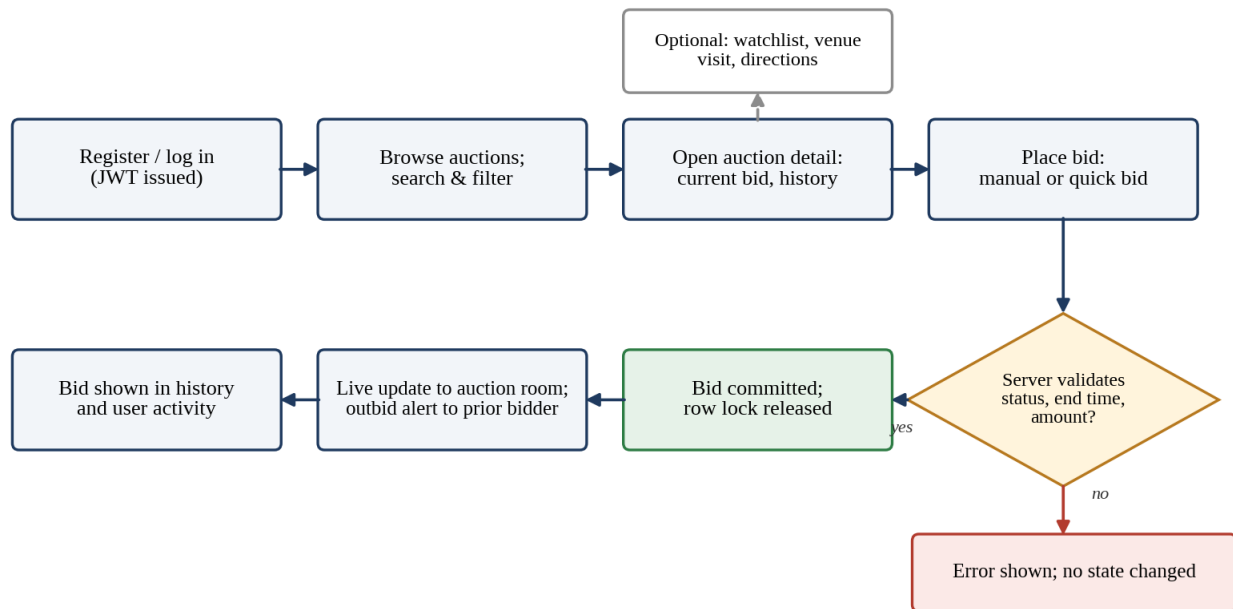


Figure 1.3 End-to-end user workflow for browsing and bidding.

If the server rejects a bid, for example because the auction has ended or the amount does not exceed the current highest bid, the user receives an error message and no state is changed. If the bid is accepted, the new highest bid appears in the bid history and in every browser that has joined the auction room. A user who is later outbid receives a notification in a personal room and can return to the auction to place a higher bid.

1.8 Administrator Workflow

Administrator workflow

The administrator workflow is not a variation of the normal user interface; it is a separate control center. The Admin navigation exposes Overview, Approvals, Products, Auctions, Users and Bids.

Approvals lists auctions that have not yet been authorized. Products provides creation and CRUD operations. Auctions provides creation, scheduling, editing and deletion. Users exposes account and role management. Bids exposes historical bid records for administrative inspection.

Access to this interface is gated twice. The login token carries the role of the user, and the `isAdmin` middleware checks that role before any administrator route is executed, so a user without the admin role is refused even if an administrator address is typed manually (Figure 1.4). Once inside, the administrator lands on the Overview, which presents dashboard statistics and a seven-day bid chart drawn with Recharts.

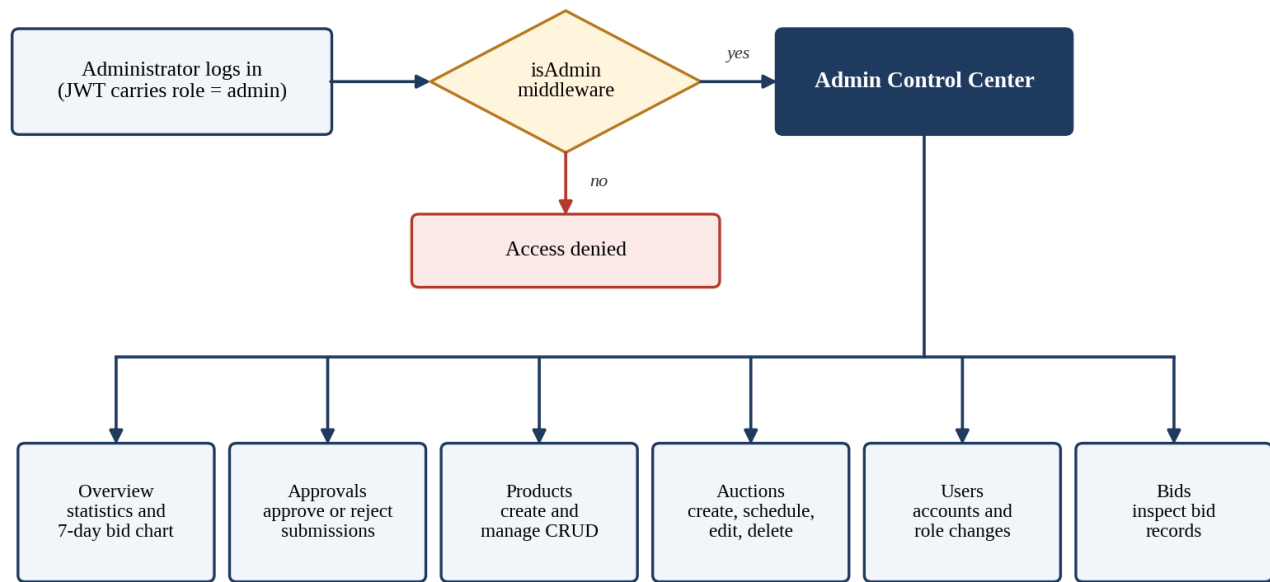


Figure 1.4 Administrator access control and Control Center modules.

A typical administrative cycle begins in Approvals, where a submitted auction is reviewed and either approved or rejected. Products are created or corrected in the catalogue, an auction is created or edited with its schedule and venue details, and the Users and Bids tabs support follow-up tasks such as changing a role or inspecting bid records when a question arises. Removal of accounts is subject to database constraints, so referential integrity is preserved even when data is deleted.

1.9 Project Significance

Database engineering significance

The strongest database-engineering contribution is the treatment of concurrent bidding as a transactional problem. The implementation does not rely on the client to decide whether a bid is valid. The server starts a transaction, locks the auction row, reads the latest value, validates the request, updates the auction, inserts the bid history and commits the unit.

This design also makes it possible to explain isolation and lost-update prevention using a concrete application rather than only a theoretical example.

Figure 1.5 shows the bid transaction in the order in which the server executes it. The row lock is taken before the current highest bid is read, so the value used for validation cannot change until the transaction ends. Other bids for the same auction wait for the lock, while bids for different auctions are unaffected because only the targeted row is locked. This keeps the critical section small and allows many auctions to be served at once.

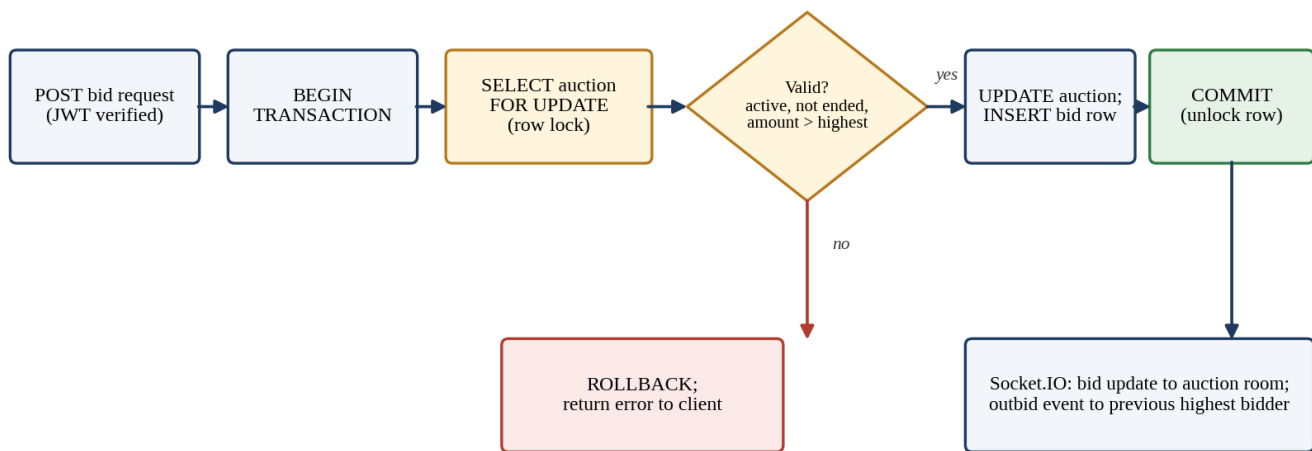


Figure 1.5 Bid transaction with row-level locking, commit/rollback and real-time broadcast.

The significance is not limited to correctness. Because the transaction either completes entirely or is rolled back, the auctions table and the bids table can never disagree about which bid is current. The composite index `idx_auction_amount` on `(auction_id, amount)` supports quick retrieval of the highest bids for an auction, and the Socket.IO broadcast follows a successful commit so that clients are not shown a bid that was later rolled back.

1.10 Expected Outcomes

Expected outcomes

The final system is expected to provide a consistent auction state, traceable bid history, authenticated access, separate administrator privileges, automated auction lifecycle management and real-time feedback.

From an academic perspective, the project demonstrates layered backend flow, relational design, SQL operations, transactions, row-level locking, indexing and a practical foundation for discussing distributed database extensions.

Deliverables

At the end of the project the deliverables are a working client application, an Express API server, a MySQL schema with demonstration data, a concurrency test script and this report. Together they allow the system to be installed on a fresh machine, populated with sample auctions and exercised by both normal users and administrators.

Success criteria

Expected outcome	Where it is demonstrated
Consistent auction state	Bid transaction (Chapter 6) and concurrency tests (Chapter 8)
Traceable bid history	bids table with the <code>idx_auction_amount</code> index; bid history views
Authenticated and separated access	JWT authentication, <code>isAdmin</code> middleware, Admin Control Center
Automated lifecycle	node-cron job activating and closing auctions; <code>auction_results</code> table
Real-time feedback	Socket.IO bid-update and outbid events
Usable interface	User and administrator features (Chapter 7) with screenshots

Learning outcomes

From a learning perspective the project is expected to build practical skills in relational schema design, writing SQL for controllers, using transactions and row locks correctly, securing a REST API with hashed passwords and signed tokens, wiring real-time events into a web interface and testing concurrent behaviour. These skills are summarised again in the conclusion of the report.

1.11 Chapter Summary

Summary

This chapter introduced Vault, its problem statement, scope, objectives, user groups and workflows. The following chapters describe the hardware and software requirements, technology choices, architecture, database design, implementation, features, testing, deployment and future scope.

Roadmap of the report

The remaining chapters follow the order of the development process. Chapter 2 lists the hardware, software and environment requirements. Chapter 3 justifies the technology stack and Chapter 4 explains how the layers cooperate. Chapter 5 presents the relational schema, Chapter 6 the implementation of the modules, authentication and concurrency control, and Chapter 7 the user and administrator features. Chapter 8 reports testing, Chapter 9 deployment, Chapter 10 challenges and limitations, Chapter 11 future enhancements and Chapter 12 concludes the report.

Three ideas from this chapter are carried through the rest of the report: the database is responsible for correctness, normal users and administrators are separated by role and by interface, and the time-driven behaviour of auctions is automated rather than manual.

Key terms used in this report

Term	Meaning in this project
Lost update	Two requests read the same bid and one write silently overwrites the other
Row lock	InnoDB lock taken on the auction row by <code>SELECT ... FOR UPDATE</code> until commit or rollback
Transaction	Group of statements that either all take effect (commit) or none do (rollback)
JWT	Signed token carrying user identity and role, sent with protected requests
Socket room	Socket.IO channel for one auction (or one user) that receives targeted events
Auction lifecycle	Submitted, approved, scheduled, active and closed, with the winner recorded

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The project is a web application and therefore does not require specialized hardware. Development can be performed on a modern laptop or desktop capable of running a browser, Node.js development servers and MySQL simultaneously.

A practical development machine should have a multi-core processor, at least 8 GB RAM, adequate SSD storage and a stable network connection. More memory is useful when the frontend, backend, database client and browser are all active together.

Because the application is web-based, end users need nothing more than a device with a current browser and a network connection. The requirements below therefore describe the development and demonstration machine, on which the client, the server and the database all run together.

Component	Requirement	Reason
Processor	Multi-core CPU	Runs the browser, Vite, Node.js and MySQL at the same time
Memory	At least 8 GB RAM	Frontend, backend, database client and browser are active together
Storage	SSD with free space	Installed packages, database files and project source
Network	Stable connection	Package installation, map tiles and real-time socket traffic
Display	Standard laptop or monitor	Dashboards, tables and map views

For a classroom demonstration all three tiers can run on a single machine, which was the configuration used during development. No graphics accelerator, dedicated server or special peripheral is required.

Notes for deployment

A production-style deployment would size memory and processor capacity according to the number of simultaneous connections, since every connected browser holds a Socket.IO connection and the MySQL connection pool is configured with 20 connections. For the scale of an academic prototype the development configuration is sufficient.

2.2 Software Requirements

Operating system and runtime

The client uses React with Vite and runs during development on the Vite development server. The server uses Node.js with Express and communicates with MySQL through mysql2/promise. The project is therefore dependent on a current Node.js/npm environment and a compatible MySQL 8.0 database.

An up-to-date browser is required for testing the interface, local storage, location permission, maps and real-time socket behaviour. The project also includes environment files for client and server configuration.

The table below lists the software that must be present before the project can be installed. All of it is available for Windows, Linux and macOS, so the project is not tied to a particular operating system.

Software	Version / type	Purpose
Node.js and npm	Current release	Runs the Express server and the Vite development server; installs packages
MySQL	8.0 with InnoDB	Relational storage, transactions and row-level locks
MySQL client	Command line or GUI	Runs schema.sql and seed.sql and inspects tables
Web browser	Up to date	Interface, local storage, location permission, maps and sockets
Git	Version control	Tracks source changes for client and server
Code editor	Any	Editing source files and environment files

Compatibility notes

The database connection pool sets an IST session time zone so that scheduled start and end times are interpreted consistently by the cron job and by the interface. On the client side, the browser must permit local storage, which holds the login token, and geolocation, which supports directions to live-auction venues. A modern browser also provides the WebSocket support on which Socket.IO relies.

2.3 Development Tools and Frameworks

Tools used in the implementation

React 18 provides the component-based frontend. Vite supplies the development/build environment. Express provides HTTP routing and middleware. Axios is used for HTTP requests. Socket.IO handles bidirectional real-time communication.

MySQL 8.0 with InnoDB provides persistent relational storage and transaction/locking support. bcrypt handles password hashing, jsonwebtoken handles authentication tokens, node-cron handles periodic auction lifecycle processing, Leaflet supports maps, and Recharts is used in the administrator dashboard.

Tool choices were guided by three criteria: support for the transactional behaviour needed by bidding, a short feedback loop during development, and a well-documented ecosystem. The table groups each tool by the layer it serves.

Tool	Layer	Contribution
React 18	Frontend	Component-based user interface
Vite	Frontend	Development server and build tooling
Axios	Frontend	HTTP client with token interceptor
Leaflet, Recharts	Frontend	Venue maps and the administrator bid chart
Express	Backend	HTTP routing and middleware
bcrypt, jsonwebtoken	Backend	Password hashing and authentication tokens
Socket.IO	Backend	Bidirectional real-time events
node-cron	Backend	Periodic auction activation and closure
MySQL 8.0 / InnoDB	Database	Persistent storage, transactions and locking

2.4 Project Dependencies

Dependency summary

Client-side dependencies are defined in `client/package.json`. Server-side dependencies are defined in `server/package.json`. The source tree also includes package-lock files so the installed dependency versions can be reproduced.

The two tables below list the main packages. Server versions are taken from `server/package.json`; client packages are managed in `client/package.json`. Both folders include lock files, so running `npm install` reproduces the same dependency tree on another machine.

Server package	Version	Role
express	^4.19.2	REST routing and middleware
mysql2	^3.11.0	MySQL connection pool with promise support
socket.io	^4.7.5	Real-time rooms and events
jsonwebtoken	^9.0.2	Issuing and verifying login tokens
bcrypt	^5.1.1	Password hashing
node-cron	^3.0.3	Scheduled auction sweep
cors, dotenv	^2.8.5, ^16.4.5	Cross-origin policy and environment loading
axios, nodemon (dev)	^1.7.2, ^3.1.4	HTTP client for scripts; automatic server restart

Client package	Role
react, react-dom (18)	Component rendering
vite	Development server and bundler
react-router	Navigation between pages
axios	Calls to the REST API
socket.io-client	Receiving live bid and notification events
leaflet, recharts, framer-motion	Maps, charts and interface transitions

The server defines the scripts `start`, `dev`, `test:concurrency` and `refresh:expo`, which launch the server, restart it on change, run the concurrency proof and refresh demonstration data respectively.

2.5 Environment Configuration

Configuration requirements

The server environment provides database connection information, JWT configuration and the client origin. The client environment provides the API and socket connection settings. Secrets are kept outside the source code through environment files.

The server environment file holds the database connection details, the JWT secret and lifetime, and the client origin used by the CORS policy. The client environment file holds the API address, which falls back to a local address on port 5000 when it is not set, and the socket address. Because these files contain secrets they should be excluded from version control. Figure 2.1 shows the order in which the environment is prepared.

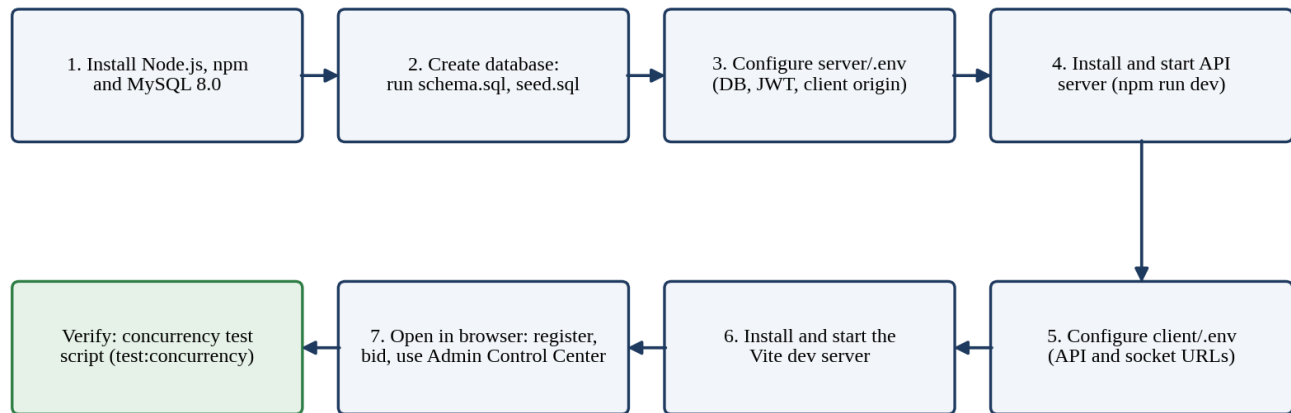


Figure 2.1 Environment preparation and start-up sequence.

Keeping configuration outside the source code means the same repository can be run in development and in a deployment simply by changing the environment files. A login token is valid for 12 hours; when it expires the client removes it and reloads once, so public pages remain usable.

2.6 Requirements Summary

Functional and technical requirements

The minimum functional requirements are authentication, auction discovery, bidding, bid history, user-specific features and administrator management. The main technical requirements are relational integrity, secure role checks, transaction-based bidding, real-time events and automated auction lifecycle processing.

The table below consolidates the requirements of this chapter and the mechanism that satisfies each of them.

Category	Requirement	Satisfied by
Functional	Authentication	Registration, login and JWT tokens
Functional	Auction discovery and bidding	Search, filters, detail view, bid form and bid history
Functional	User-specific features	Watchlist, recommendations, activity, visits, notifications
Functional	Administrator management	Separate Control Center with six modules
Technical	Relational integrity	Seven-table InnoDB schema with keys and constraints
Technical	Secure role checks	JWT middleware and isAdmin guard
Technical	Concurrency-safe bidding	Transaction with SELECT ... FOR UPDATE
Technical	Real-time events and automation	Socket.IO rooms and node-cron job

This chapter has defined the machine, software and configuration needed to build and run Vault. The next chapter describes the technology stack that fulfils these requirements.

Non-functional requirements

Besides the functional list above, the system must provide consistency, so that concurrent bids never corrupt auction state; security, through hashed passwords, signed tokens and role checks; timeliness, so that bid updates and auction closure reach users promptly; maintainability, through a layered folder structure and environment-based configuration; and portability, so that the project runs on any machine with the listed software.

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-End

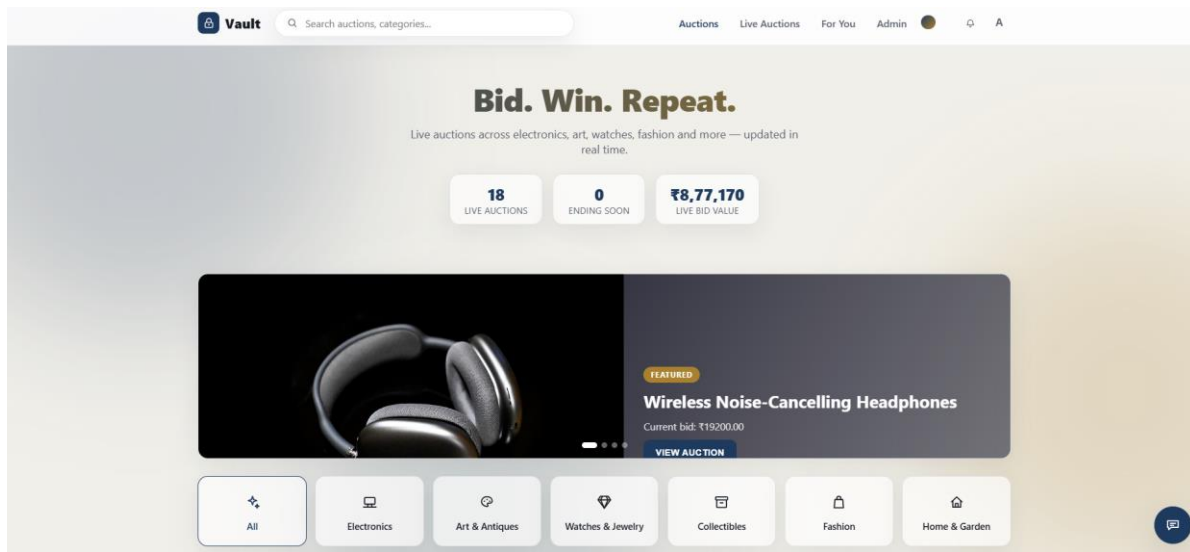


Figure 3.1 Vault home page: live statistics and featured auction.

3.2 Front-End State and API Communication

Axios centralizes requests from the client to the REST backend. The authentication token is attached to protected requests. Client-side state is used for filters, auction data, notifications and user-specific views. The UI is organized around the user journey rather than a single large page.

```
• client/src/api/axios.js
1 import axios from 'axios';
2
3 const api = axios.create({
4   baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
5 });
6
7 api.interceptors.request.use((config) => {
8   const token = localStorage.getItem('token');
9   if (token) config.headers.Authorization = `Bearer ${token}`;
10  return config;
11 });
12
13 // A login token expires after 12h. Drop it (and reload once) so public pages keep working
14 // instead of failing with 401 "Invalid or expired token".
15 api.interceptors.response.use(
16   (res) => res,
17   (err) => {
18     if (err.response?.status === 401 && localStorage.getItem('token') &&
19       /expired|invalid/i.test(err.response?.data?.error || '')) {
20       localStorage.removeItem('token');
21       localStorage.removeItem('user');
22       window.location.reload();
23     }
24     return Promise.reject(err);
25   }
26 );
27 export default api;
```

3.3 Back-End

Node.js and Express form the application server. Routes define API endpoints, middleware validates authentication and administrator access, and controllers implement business logic. The MySQL pool provides database access. Socket.IO is attached to the same server so REST operations and real-time events can share application state.

```
• server/app.js
1  require('dotenv').config();
2  const express = require('express');
3  const cors = require('cors');
4
5  const authRoutes = require('./routes/authRoutes');
6  const productRoutes = require('./routes/productRoutes');
7  const auctionRoutes = require('./routes/auctionRoutes');
8  const adminRoutes = require('./routes/adminRoutes');
9  const watchlistRoutes = require('./routes/watchlistRoutes');
10 const recommendationRoutes = require('./routes/recommendationRoutes');
11 const userRoutes = require('./routes/userRoutes');
12 const visitRoutes = require('./routes/visitRoutes');
13
14 const app = express();
15
16 app.use(cors({ origin: process.env.CLIENT_ORIGIN || '*' }));
17 app.use(express.json());
18
19 app.get('/api/health', (req, res) => res.json({ status: 'ok' }));
20
21 app.use('/api/auth', authRoutes);
22 app.use('/api/products', productRoutes);
23 app.use('/api/auctions', auctionRoutes);
24 app.use('/api/admin', adminRoutes);
25 app.use('/api/watchlist', watchlistRoutes);
26 app.use('/api/recommendations', recommendationRoutes);
27 app.use('/api/users', userRoutes);
28 app.use('/api/visits', visitRoutes);
29
30 app.use((req, res) => res.status(404).json({ error: 'Not found' }));
31
32 module.exports = app;
```

3.4 Database

MySQL 8.0 with InnoDB is used because bidding requires transactions and row-level locking. The principal tables are users, products, auctions, bids, auction_results, watchlist and visits. Foreign keys preserve relationships between these entities.

Table	Primary key	Foreign keys	Notes
users	id	—	email UNIQUE; role ENUM(user, admin)
products	id	created_by → users.id (SET NULL)	category, is_certified
auctions	id	product_id → products.id; current_highest_bidder_id → users.id	status, approval_status, venue_* columns
bids	id	auction_id → auctions.id; user_id → users.id	INDEX idx_auction_amount (auction_id, amount)
auction_results	auction_id	auction_id, winning_bid_id, winner_id	One result row per auction
watchlist	(user_id, auction_id)	user_id, auction_id (CASCADE)	Saved auctions
visits	(user_id, auction_id)	user_id, auction_id (CASCADE)	Scheduled venue visits

3.5 Authentication Libraries

Passwords are hashed with bcrypt. JWT tokens carry the user identity and role so the backend can authenticate requests and authorize administrator operations. The token is attached to subsequent API requests by the client.

```
• server/package.json
1 {
2   "name": "auction-platform-server",
3   "version": "1.0.0",
4   "description": "Online Auction & Bidding Platform - backend with real-time concurrency handling",
5   "main": "server.js",
6   "type": "commonjs",
7   "scripts": {
8     "start": "node server.js",
9     "dev": "nodemon server.js",
10    "test:concurrency": "node test/concurrentBidTest.js",
11    "refresh:expo": "node scripts/refreshExpo.js"
12  },
13  "dependencies": {
14    "bcrypt": "^5.1.1",
15    "cors": "^2.8.5",
16    "dotenv": "^16.4.5",
17    "express": "^4.19.2",
18    "jsonwebtoken": "^9.0.2",
19    "mysql2": "^3.11.0",
20    "node-cron": "^3.0.3",
21    "socket.io": "^4.7.5"
22  },
23  "devDependencies": {
24    "axios": "^1.7.2",
25    "nodemon": "^3.1.4"
26  }
27 }
```

3.6 Real-Time Communication

Socket.IO provides auction rooms and personal notification rooms. After a successful bid, the server emits a bid update to the auction room and can notify the previous highest bidder that they have been outbid.

```
• server/sockets/index.js
1 function registerSocketHandlers(io) {
2   io.on('connection', (socket) => {
3     socket.on('join_auction', (auctionId) => {
4       socket.join(`auction_${auctionId}`);
5     });
6
7     socket.on('leave_auction', (auctionId) => {
8       socket.leave(`auction_${auctionId}`);
9     });
10
11    // Personal room for targeted notifications (e.g. "you've been outbid").
12    socket.on('identify', (userId) => {
13      if (userId) socket.join(`user_${userId}`);
14    });
15
16    socket.on('disconnect', () => {
17      // no-op; rooms are cleaned up automatically
18    });
19  });
20 }
21
22 module.exports = registerSocketHandlers;
```

3.7 Scheduled Processing

The closeAuctions job runs periodically. It activates scheduled auctions when their start time arrives and closes active auctions whose end time has passed. Closure records the winner in auction_results and emits the corresponding event.

```
• server/jobs/closeAuctions.js (lines 94-104)
94 function startAuctionCloseJob(io) {
95   // Runs every 30 seconds - frequent enough for a demo, cheap enough in general.
96   cron.schedule('*/*30 * * * *', async () => {
97     try {
98       await activateStartedAuctions(io);
99       await closeExpiredAuctions(io);
100    } catch (err) {
101      console.error('Auction sweep failed:', err);
102    }
103  });
104 }
```

3.8 Mapping Services

The live-auctions workflow uses Leaflet for map rendering and the OpenStreetMap ecosystem for map tiles and supporting location/route services. The UI can request location access, display a venue and calculate route information.

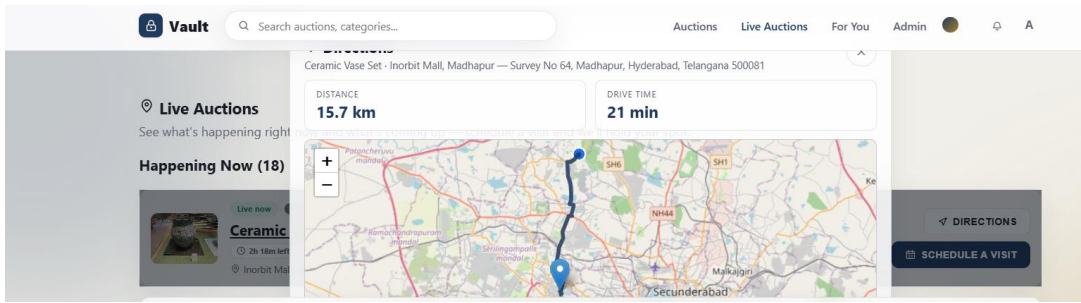


Figure 3.8 Route and drive-time view rendered with Leaflet / OpenStreetMap.

3.9 Charts and UI Motion

Recharts is used for the administrator's seven-day bid chart. Framer Motion is used for interface transitions. These libraries support presentation and interaction without changing the authoritative database state.

```
• client/src/pages/AdminDashboard.jsx (lines 44-52)
44     <ResponsiveContainer>
45       <LineChart data={chartData}>
46         <CartesianGrid strokeDasharray="3 3" stroke="var(--border)" />
47         <XAxis dataKey="day" stroke="var(--text-muted)" fontSize={12} />
48         <YAxis stroke="var(--text-muted)" fontSize={12} allowDecimals={false} />
49         <Tooltip contentStyle={{ background: 'var(--surface-solid)', border: '1px solid var(--border)',
borderRadius: 10 }} />
50         <Line type="monotone" dataKey="bids" stroke="var(--accent)" strokeWidth={2} dot={{ r: 3, fill: 'var(--
accent)' }} activeDot={{ r: 5 }} />
51       </LineChart>
52     </ResponsiveContainer>
```

3.10 Project Structure

The repository is divided into client and server. The server is further organized into config, controllers, middleware, routes, sockets, jobs, test, data and SQL files. The client contains components, context, pages, API helpers and styles.

Path	Contents
server/config/db.js	MySQL connection pool (20 connections, IST session time zone)
server/controllers/	auth, auction, bid, product, admin, watchlist, visit, recommendation logic
server/middleware/	auth.js (JWT) and isAdmin.js (role guard)
server/routes/	Express route definitions per resource
server/sockets/, server/jobs/	Socket.IO rooms; node-cron activate/close job
server/test/	concurrentBidTest.js concurrency proof script
server/schema.sql, seed.sql	DDL and demonstration data
client/src/pages/, components/	React pages (Auctions, Live Auctions, Admin...) and reusable UI
client/src/context/, api/	Auth, Socket, Notification, Theme contexts; Axios instance

3.11 Technology Stack Summary

The stack is therefore a full client-server relational application: React/Vite at the interface, Node/Express in the application layer, MySQL/InnoDB for persistence, Socket.IO for live communication, node-cron for lifecycle automation and Leaflet/OpenStreetMap-related services for venue directions.

Technology	Layer	Role
React 18 / Vite	Frontend	Components, routing, development server
Node.js / Express	Backend	REST routes, middleware, controllers
MySQL 8.0 / InnoDB	Database	Relational persistence, transactions, row locks
Socket.IO	Real-time	Bid and outbid events
bcrypt / JWT	Security	Password hashing and authentication
node-cron	Automation	Auction activation and closure
Leaflet / map services	Location	Venue maps and routes
Recharts / Framer Motion	UI support	Admin chart and transitions

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture

CLIENT / BROWSER

React + Vite + Router + Context + Axios

↓ REST requests

↓ Socket.IO events

EXPRESS APPLICATION SERVER

Routes → Authentication / Admin Middleware → Controllers



MYSQL 8.0 / INNODB

users • products • auctions • bids • results • watchlist • visits

Background node-cron job → activates / closes auctions

The system follows a layered architecture. The browser presents the React application. Normal operations travel through REST requests to Express routes and controllers. Protected operations pass through JWT and administrator middleware. Controllers use the MySQL pool for persistence. A parallel Socket.IO connection carries real-time events. A background cron process handles auction lifecycle transitions.

4.2 Frontend Layer

The frontend contains public pages such as auction listings, auction detail, login, registration, recommendations, live auctions and activity. Reusable components support filters, cards, bid forms, countdowns, maps, notifications, chatbot interactions and administrator tables.

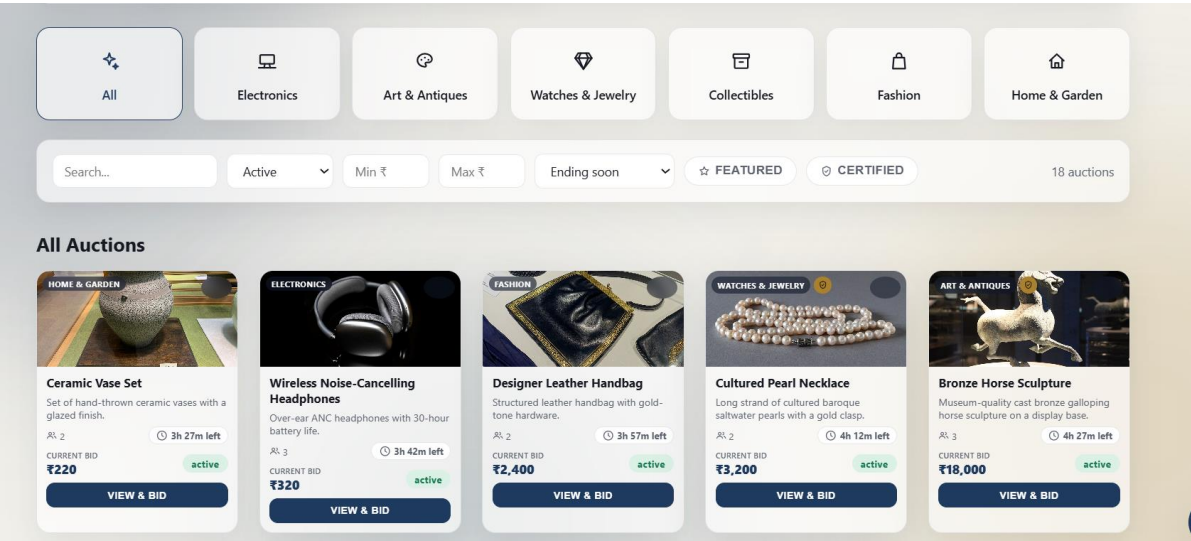


Figure 4.2 Frontend layer: category tiles, filter bar and auction cards.

4.3 Backend and Database Layers

Routes map HTTP methods and paths to controller functions. Controllers validate input, execute queries and return JSON. The database stores the durable state. The bid transaction makes the auction row the synchronization point for concurrent requests.

```
• server/routes/auctionRoutes.js
1  const express = require('express');
2  const router = express.Router();
3  const authenticate = require('../middleware/auth');
4  const isAdmin = require('../middleware/isAdmin');
5  const { listAuctions, getAuction, createAuction, updateAuction, closeAuctionNow, deleteAuction, getResult } =
    require('../controllers/auctionController');
6  const { placeBid, getBidHistory } = require('../controllers/bidController');
7
8  router.get('/', listAuctions);
9  router.get('/:id', getAuction);
10 router.post('/', authenticate, isAdmin, createAuction);
11 router.put('/:id', authenticate, isAdmin, updateAuction);
12 router.post('/:id/close', authenticate, isAdmin, closeAuctionNow);
13 router.delete('/:id', authenticate, isAdmin, deleteAuction);
14 router.get('/:id/result', getResult);
15
16 router.post('/:id/bids', authenticate, placeBid);
17 router.get('/:id/bids', getBidHistory);
18
19 module.exports = router;
```

4.4 Deployment Architecture

During local development the Vite client runs on port 5173 and the Node backend on port 5000. MySQL runs locally or on an accessible database host. Environment variables connect the layers. The same separation can be preserved when moving to hosted infrastructure.

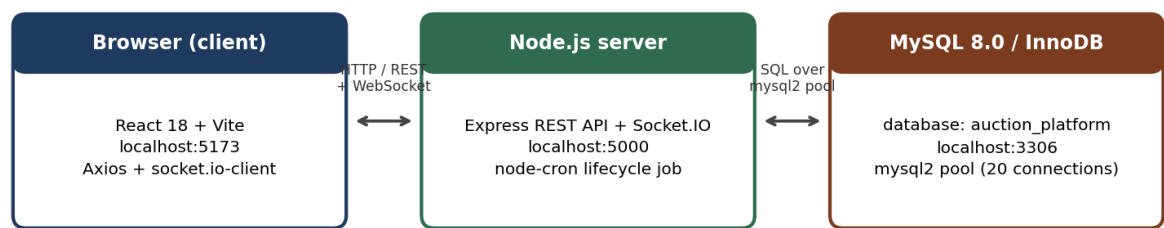


Figure 4.2 Local development deployment: three tiers connected through environment-configured URLs

Figure 4.4 Local deployment architecture.

CHAPTER -5 DESIGN

5.1 ER Diagram and Relational Design

Table	Key attributes	Purpose
users	id PK, name, email, password_hash, role	Referenced by bids, products, visits, watchlist
products	id PK, name, description, category, is_certified, created_by FK	Auctionable product
auctions	id PK, product_id FK, prices, times, status, highest bidder FK	Auction state
bids	id PK, auction_id FK, user_id FK, amount, created_at	Bid history
auction_results	auction_id PK/FK, winning_bid_id FK, winner_id FK	Final result
watchlist	user_id + auction_id composite PK	Saved auctions
visits	user_id + auction_id composite PK	Scheduled venue visits

The relational design separates accounts, products, auctions, bids and supporting records. users is referenced by products, auctions through the highest bidder, bids, auction_results, watchlist and visits. products are auctionable items. auctions hold the current state and timing. bids preserve the history. auction_results stores the finalized winner. watchlist and visits use composite keys so duplicate user-auction pairs are prevented.

Figure 5.1 Entity-Relationship diagram of the Vault database (auction_platform)

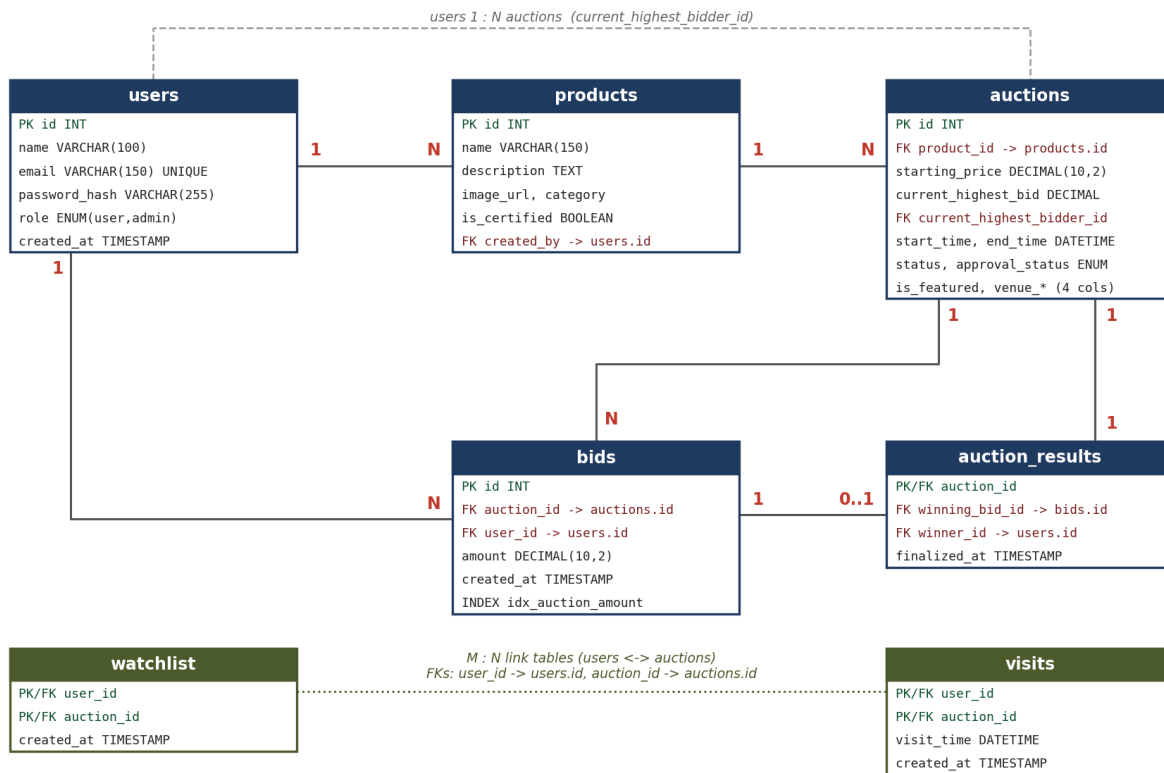


Figure 5.1 ER diagram of the auction_platform database.

CHAPTER -6 IMPLEMENTATION

6.1 Authentication Module

```
• server/controllers/authController.js
1  const bcrypt = require('bcrypt');
2  const jwt = require('jsonwebtoken');
3  const pool = require('../config/db');
4
5  async function register(req, res) {
6    try {
7      const { name, email, password } = req.body;
8      if (!name || !email || !password) {
9        return res.status(400).json({ error: 'name, email, and password are required' });
10     }
11
12     const [existing] = await pool.query('SELECT id FROM users WHERE email = ?', [email]);
13     if (existing.length > 0) {
14       return res.status(409).json({ error: 'Email already registered' });
15     }
16
17     const passwordHash = await bcrypt.hash(password, 10);
18     const [result] = await pool.query(
19       'INSERT INTO users (name, email, password_hash, role) VALUES (?, ?, ?, ?)',
20       [name, email, passwordHash, 'user']
21     );
22
23     const token = jwt.sign(
24       { id: result.insertId, name, role: 'user' },
25       process.env.JWT_SECRET,
26       { expiresIn: '12h' }
27     );
28
29     res.status(201).json({ token, user: { id: result.insertId, name, email, role: 'user' } });
30   } catch (err) {
31     console.error(err);
32     res.status(500).json({ error: 'Registration failed' });
33   }
34 }
35
36 async function login(req, res) {
37   try {
38     const { email, password } = req.body;
39     if (!email || !password) {
40       return res.status(400).json({ error: 'email and password are required' });
41     }
42
43     const [rows] = await pool.query('SELECT * FROM users WHERE email = ?', [email]);
44     if (rows.length === 0) {
45       return res.status(401).json({ error: 'Invalid credentials' });
46     }
47
48     const user = rows[0];
49     const match = await bcrypt.compare(password, user.password_hash);
50     if (!match) {
51       return res.status(401).json({ error: 'Invalid credentials' });
52     }
53
54     const token = jwt.sign(
55       { id: user.id, name: user.name, role: user.role },
56       process.env.JWT_SECRET,
57       { expiresIn: '12h' }
58     );
59
60     res.json({
61       token,
```

```
62     user: { id: user.id, name: user.name, email: user.email, role: user.role },
63   });
64 } catch (err) {
65   console.error(err);
66   res.status(500).json({ error: 'Login failed' });
67 }
68 }
69
70 module.exports = { register, login };
```

Registration checks required fields and email uniqueness, hashes the password and creates a user. Login verifies the password hash and returns a signed JWT containing identity and role information. The client keeps the token and uses it for protected requests.

6.2 Authentication Middleware

The authentication middleware extracts the Bearer token, verifies the JWT secret and places the decoded user information on the request. This provides a consistent boundary before protected controllers execute.

```
• server/middleware/auth.js
1  const jwt = require('jsonwebtoken');
2
3  function authenticate(req, res, next) {
4    const header = req.headers.authorization;
5    if (!header || !header.startsWith('Bearer ')) {
6      return res.status(401).json({ error: 'No token provided' });
7    }
8    const token = header.split(' ')[1];
9    try {
10     const payload = jwt.verify(token, process.env.JWT_SECRET);
11     req.user = payload; // { id, name, role }
12     next();
13   } catch (err) {
14     return res.status(401).json({ error: 'Invalid or expired token' });
15   }
16 }
17
18 function optionalAuthenticate(req, res, next) {
19   const header = req.headers.authorization;
20   if (!header || !header.startsWith('Bearer ')) return next();
21   const token = header.split(' ')[1];
22   try {
23     req.user = jwt.verify(token, process.env.JWT_SECRET);
24   } catch {
25     // Invalid/expired token on an optional route: proceed as a guest.
26   }
27   next();
28 }
29
30 module.exports = authenticate;
31 module.exports.optionalAuthenticate = optionalAuthenticate;
```

6.3 Administrator Authorization

The administrator middleware is a second authorization layer. Authentication alone does not grant access to administrative endpoints. The middleware checks the authenticated role and rejects requests that are not authorized as admin.

```
• server/middleware/isAdmin.js
1 function isAdmin(req, res, next) {
2   if (!req.user || req.user.role !== 'admin') {
3     return res.status(403).json({ error: 'Admin access required' });
4   }
5   next();
6 }
7
8 module.exports = isAdmin;
```

```
• server/routes/adminRoutes.js (lines 7-13)
7 router.get('/users', authenticate, isAdmin, listUsers);
8 router.patch('/users/:id/role', authenticate, isAdmin, updateUserRole);
9 router.delete('/users/:id', authenticate, isAdmin, deleteUser);
10 router.get('/bids', authenticate, isAdmin, listAllBids);
11 router.get('/stats', authenticate, isAdmin, getStats);
12 router.get('/auctions/pending', authenticate, isAdmin, listPendingAuctions);
13 router.patch('/auctions/:id/approval', authenticate, isAdmin, setAuctionApproval);
```

6.4 Auction Discovery Module

The auction controller joins auction, product and bid data. It returns bid count and bidder count and exposes listing and detail endpoints. Public listings are intended to show meaningful auction state while administrator operations remain protected.

```
• server/controllers/auctionController.js (lines 3-19)
3  async function listAuctions(req, res) {
4    try {
5      const [rows] = await pool.query(`
6        SELECT a.*, p.name AS product_name, p.description AS product_description, p.image_url, p.category,
6        p.is_certified,
7          COUNT(b.id) AS bid_count, COUNT(DISTINCT b.user_id) AS bidder_count
8        FROM auctions a
9        JOIN products p ON p.id = a.product_id
10       LEFT JOIN bids b ON b.auction_id = a.id
11       GROUP BY a.id
12       ORDER BY a.start_time DESC
13     `);
14     res.json(rows);
15   } catch (err) {
16     console.error(err);
17     res.status(500).json({ error: 'Failed to fetch auctions' });
18   }
19 }
```

6.5 Product Management Module

Products are managed separately from auctions. An administrator can create products with name, description, image URL, category and certification state, then edit or delete them. The separation allows a product record to be reused by an auction record.

```
• server/controllers/productController.js (lines 26-40)
26 async function createProduct(req, res) {
27   try {
28     const { name, description, image_url, category, is_certified } = req.body;
29     if (!name) return res.status(400).json({ error: 'name is required' });
30
31     const [result] = await pool.query(
32       'INSERT INTO products (name, description, image_url, category, is_certified, created_by) VALUES (?, ?, ?, ?,
33       ?, ?)',
34       [name, description || null, image_url || null, category || null, !!is_certified, req.user.id]
35     );
36     res.status(201).json({ id: result.insertId, name, description, image_url, category, is_certified: !!is_certified
37     });
38   } catch (err) {
39     console.error(err);
40     res.status(500).json({ error: 'Failed to create product' });
41   }
42 }
```

6.6 Bidding Module

The bid controller is the critical implementation. It obtains a database connection, begins a transaction, locks the target auction row with `SELECT ... FOR UPDATE`, validates the amount and timing, updates the auction, inserts the bid history, commits and releases the connection. Rollback is used on failure.

```
• server/controllers/bidController.js (lines 15-79)
15 async function placeBidTransaction(auctionId, userId, amount) {
16   const connection = await pool.getConnection();
17   try {
18     await connection.beginTransaction();
19
20     // Row lock: any other transaction trying to SELECT ... FOR UPDATE the
21     // same auction row will block here until this transaction commits or
22     // rolls back.
23     const [rows] = await connection.query(
24       'SELECT current_highest_bid, current_highest_bidder_id, status, end_time FROM auctions WHERE id = ? FOR
UPDATE',
25       [auctionId]
26     );
27
28     if (rows.length === 0) {
29       await connection.rollback();
30       return { success: false, status: 404, error: 'Auction not found' };
31     }
32
33     const auction = rows[0];
34
35     if (auction.status !== 'active') {
36       await connection.rollback();
37       return { success: false, status: 400, error: 'Auction is not active' };
38     }
39
40     if (new Date() > new Date(auction.end_time)) {
41       await connection.rollback();
42       return { success: false, status: 400, error: 'Auction has already ended' };
43     }
44
45     if (Number(amount) <= Number(auction.current_highest_bid)) {
46       await connection.rollback();
47       return {
48         success: false,
49         status: 409,
50         error: `Bid must be higher than current highest bid (${auction.current_highest_bid})`,
51       };
52     }
53
54     await connection.query(
55       'UPDATE auctions SET current_highest_bid = ?, current_highest_bidder_id = ? WHERE id = ?',
56       [amount, userId, auctionId]
57     );
58
59     const [insertResult] = await connection.query(
60       'INSERT INTO bids (auction_id, user_id, amount) VALUES (?, ?, ?)',
61       [auctionId, userId, amount]
62     );
63
64     await connection.commit();
65
66     return {
67       success: true,
68       bidId: insertResult.insertId,
69       auctionId: Number(auctionId),
70       amount: Number(amount),
71       previousBidderId: auction.current_highest_bidder_id || null,
72     };

```

```
73     } catch (err) {  
74         await connection.rollback();  
75         throw err;  
76     } finally {  
77         connection.release();  
78     }  
79 }
```

6.7 Deadlock Retry Handling

Because transactional systems can encounter deadlocks under concurrency, the implementation includes retry behaviour around the critical bid workflow. The intent is to retry transient deadlock failures rather than expose an inconsistent state to the user.

```
• server/controllers/bidController.js (lines 87-99)
87 async function placeBidWithRetry(auctionId, userId, amount, retries = 3, attempt = 1) {
88   try {
89     return await placeBidTransaction(auctionId, userId, amount);
90   } catch (err) {
91     const isDeadlock = err.code === 'ER_LOCK_DEADLOCK' || err.errno === 1213;
92     if (isDeadlock && retries > 0) {
93       const backoff = 50 * attempt + Math.random() * 100;
94       await sleep(backoff);
95       return placeBidWithRetry(auctionId, userId, amount, retries - 1, attempt + 1);
96     }
97     throw err;
98   }
99 }
```

6.8 Real-Time Module

Socket.IO organizes auction-specific rooms and personal notification rooms. Successful bid operations emit the updated bid state. Outbid events are directed to the previous highest bidder. This means the UI can update without polling the database for every visible change.

```
• server/controllers/bidController.js (lines 121-147)
121   const io = req.app.get('io');
122   if (io) {
123     io.emit('bid_update', {
124       auctionId: Number(auctionId),
125       currentHighestBid: result.amount,
126       bidderId: userId,
127       bidderName: req.user.name,
128       time: new Date().toISOString(),
129     });
130
131     // Targeted real-time alert to whoever held the top bid before this one.
132     if (result.previousBidderId && result.previousBidderId !== userId) {
133       const [[product]] = await pool.query(
134         `SELECT p.name AS product_name FROM auctions a
135          JOIN products p ON p.id = a.product_id WHERE a.id = ?`,
136         [auctionId]
137       );
138       io.to(`user_${result.previousBidderId}`).emit('outbid', {
139         auctionId: Number(auctionId),
140         productName: product?.product_name || 'an item',
141         newAmount: result.amount,
142         outbidBy: req.user.name,
143       });
144     }
145   }
146
147   res.status(201).json(result);
```

6.9 Auction Lifecycle Job

The scheduled job checks due auctions and expired active auctions. Activation changes scheduled records into active records. Closure finalizes the auction and records the winner. The job can also emit an auction-closed event so connected clients receive the state transition.

```
• server/jobs/closeAuctions.js (lines 9-60)
 9  async function closeAuctionById(auctionId, io) {
10    const connection = await pool.getConnection();
11    try {
12      await connection.beginTransaction();
13
14      const [rows] = await connection.query(
15        `SELECT id, current_highest_bid, current_highest_bidder_id, status
16         FROM auctions WHERE id = ? FOR UPDATE`,
17        [auctionId]
18      );
19      if (rows.length === 0 || rows[0].status !== 'active') {
20        await connection.rollback();
21        return false;
22      }
23      const auction = rows[0];
24
25      await connection.query(`UPDATE auctions SET status = 'closed' WHERE id = ?`, [auction.id]);
26
27      let winningBidId = null;
28      if (auction.current_highest_bidder_id) {
29        const [bidRows] = await connection.query(
30          `SELECT id FROM bids WHERE auction_id = ? ORDER BY amount DESC, created_at ASC LIMIT 1`,
31          [auction.id]
32        );
33        winningBidId = bidRows[0]?.id || null;
34      }
35
36      await connection.query(
37        `INSERT INTO auction_results (auction_id, winning_bid_id, winner_id)
38         VALUES (?, ?, ?)
39         ON DUPLICATE KEY UPDATE winning_bid_id = VALUES(winning_bid_id), winner_id = VALUES(winner_id)`,
40        [auction.id, winningBidId, auction.current_highest_bidder_id]
41      );
42
43      await connection.commit();
44
45      if (io) {
46        io.emit('auction_closed', {
47          auctionId: auction.id,
48          winnerId: auction.current_highest_bidder_id,
49          winningBid: auction.current_highest_bid,
50        });
51      }
52      return true;
53    } catch (err) {
54      await connection.rollback();
55      console.error(`Failed to close auction ${auctionId}:`, err);
56      throw err;
57    } finally {
58      connection.release();
59    }
60  }
```

6.10 Watchlist, Recommendations and Visits

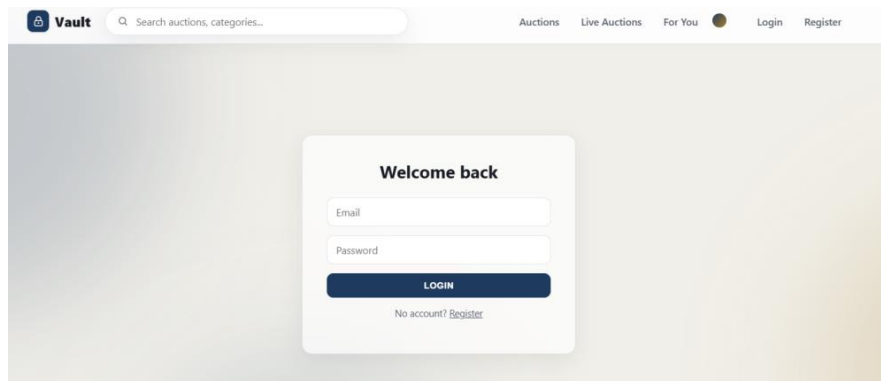
User-specific controllers support watchlist entries, activity and recommendations. The visits workflow records a user's planned venue visit for a live auction and integrates the location interface. These features extend the platform beyond the core bid transaction while remaining connected to the same relational model.

```
• server/controllers/watchlistController.js (lines 24-36)
24 async function addToWatchlist(req, res) {
25   try {
26     const { auctionId } = req.params;
27     await pool.query(
28       'INSERT IGNORE INTO watchlist (user_id, auction_id) VALUES (?, ?)',
29       [req.user.id, auctionId]
30     );
31     res.status(201).json({ watching: true });
32   } catch (err) {
33     console.error(err);
34     res.status(500).json({ error: 'Failed to add to watchlist' });
35   }
36 }
```

```
• server/controllers/visitController.js (lines 22-38)
22 async function scheduleVisit(req, res) {
23   try {
24     const { auctionId } = req.params;
25     const { visit_time } = req.body;
26     if (!visit_time) return res.status(400).json({ error: 'visit_time is required' });
27
28     await pool.query(
29       `INSERT INTO visits (user_id, auction_id, visit_time) VALUES (?, ?, ?)
30       ON DUPLICATE KEY UPDATE visit_time = VALUES(visit_time)`,
31       [req.user.id, auctionId, visit_time]
32     );
33     res.status(201).json({ scheduled: true, visit_time });
34   } catch (err) {
35     console.error(err);
36     res.status(500).json({ error: 'Failed to schedule visit' });
37   }
38 }
```

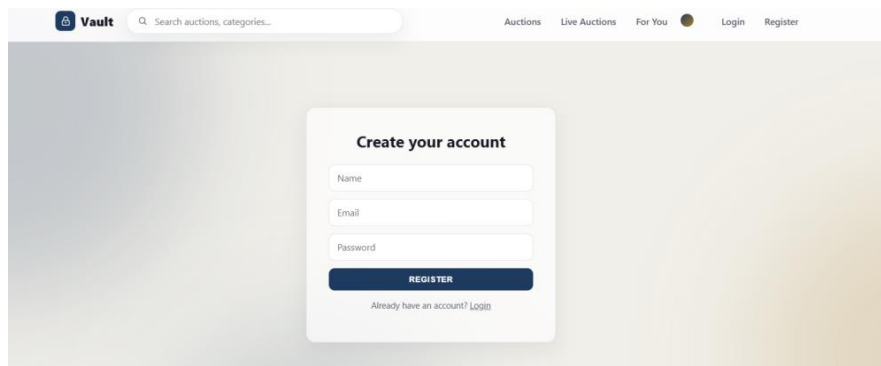
CHAPTER 7 - FEATURES

7.1 User Features — Authentication



The login page features a header with the 'Vault' logo, a search bar, and navigation links for 'Auctions', 'Live Auctions', 'For You', 'Login', and 'Register'. The main content area contains a 'Welcome back' modal with input fields for 'Email' and 'Password', a 'LOGIN' button, and a link to 'Register' for users without an account.

Figure 7.1a User login page.



The registration page features the same header as the login page. The main content area contains a 'Create your account' modal with input fields for 'Name', 'Email', and 'Password', a 'REGISTER' button, and a link to 'Login' for users who already have an account.

Figure 7.1b User registration page.

7.2 User Features — Auction Discovery

The main auction interface provides category selection, search, status, minimum and maximum price, ending-soon sorting, featured and certified filters. Cards display the product, current bid, remaining time, bidder information and a view-and-bid action.

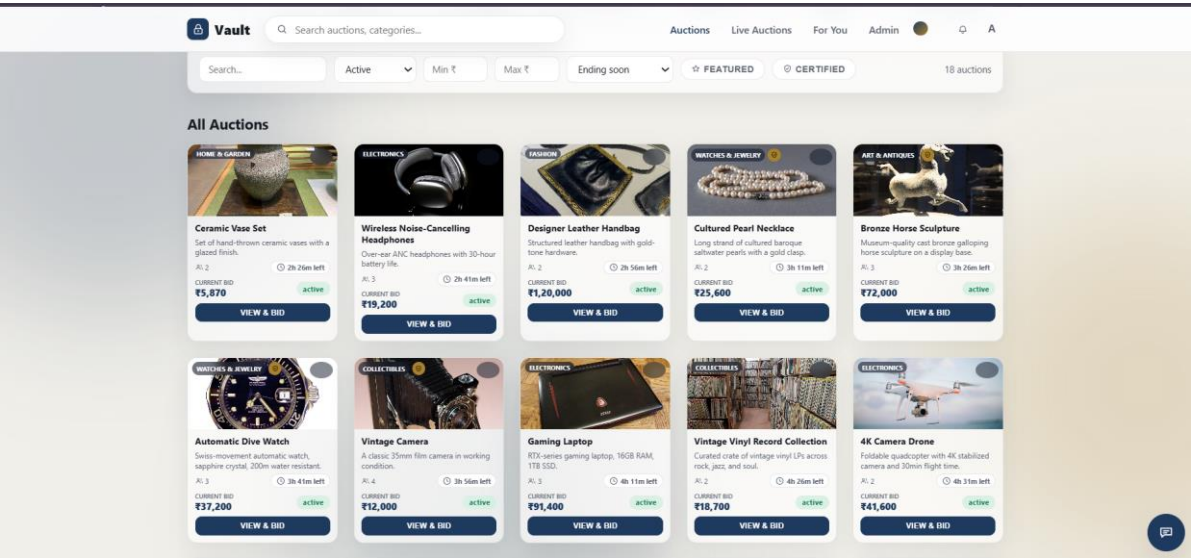


Figure 7.2a Auction listing with search, status, price and sort filters.

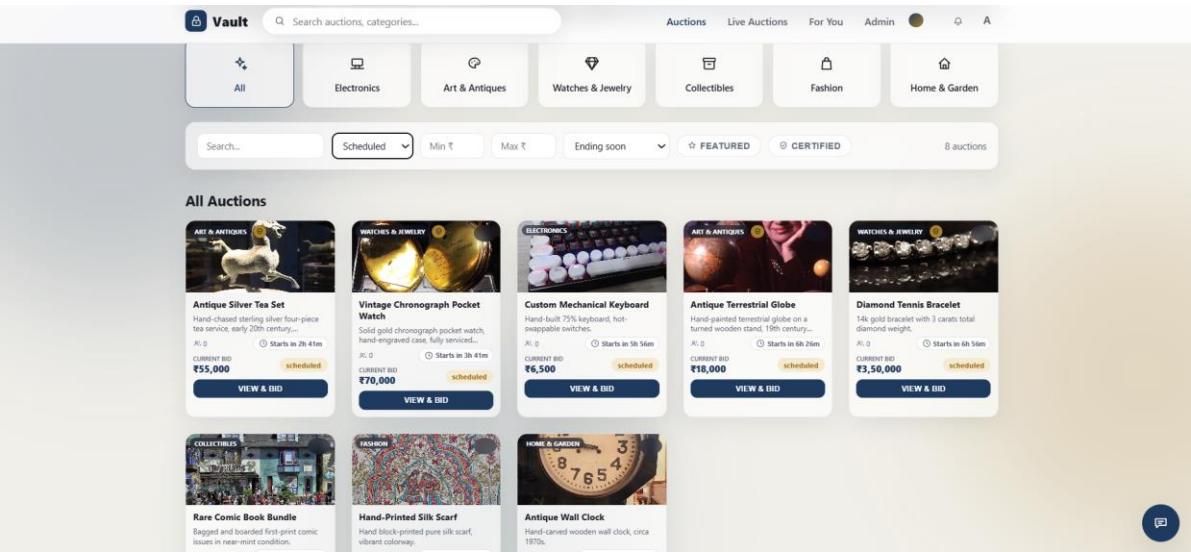


Figure 7.2b Scheduled auctions with upcoming start times.

7.3 User Features — Bidding and History

A user can open an auction and place a bid. Quick bid controls can add percentage-based increments. The current bid and bid history provide feedback. The authoritative validation happens on the backend, not in the browser.

```
• client/src/components/BidForm.jsx (lines 11-23)
11  async function submitAmount(value) {
12    setError('');
13    setSubmitting(true);
14    try {
15      await api.post(`/auctions/${auction.id}/bids`, { amount: Number(value) });
16      setAmount('');
17      onBidPlaced?.();
18    } catch (err) {
19      setError(err.response?.data?.error || 'Failed to place bid');
20    } finally {
21      setSubmitting(false);
22    }
23  }
```

7.4 User Features — Watchlist and Recommendations

The platform provides personal features for saving auctions and viewing recommendations. These functions are backed by user-specific database records and controller endpoints.

```
• client/src/pages/Recommendations.jsx (lines 13-18)
13  function load() {
14    setFailed(false);
15    api.get('/recommendations').then((res) => setItems(res.data)).catch(() => setFailed(true));
16  }
17
18  useEffect(() => { load(); }, []);
```

7.5 User Features — Live Auctions and Directions

Live Auctions presents currently active venue-based auctions. Each entry exposes Directions and Schedule a Visit actions. Location permission can be requested by the browser and the interface can display a map and route information.

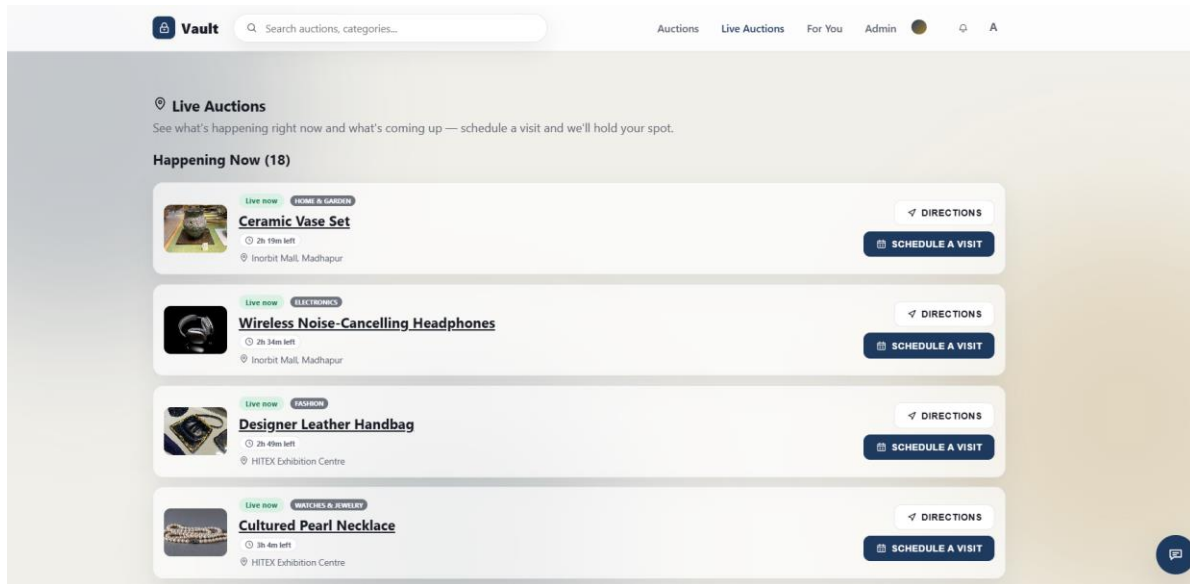


Figure 7.5a Live Auctions page listing venue-based auctions.

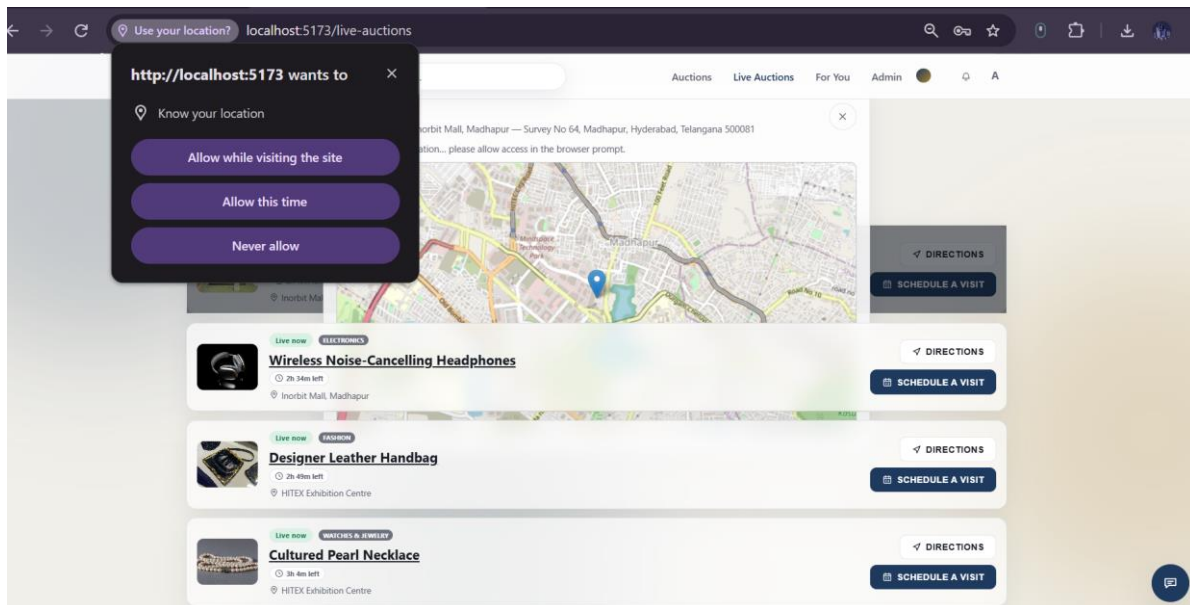


Figure 7.5b Browser location permission and venue map.

7.6 User Features — Visit Scheduling

The visit workflow lets the user select a date/time for a venue visit and confirm or cancel the appointment. The visit record is associated with the user and auction, preventing duplicate user-auction visit records through the composite key design.

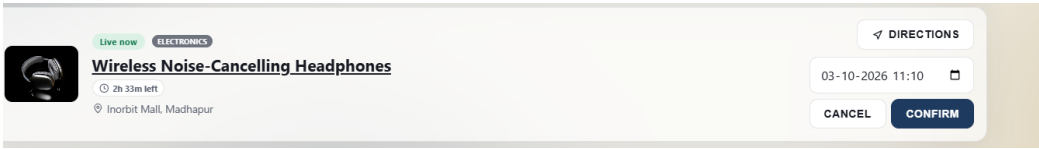


Figure 7.6 Visit scheduling with date/time picker, Confirm and Cancel.

7.7 User Features — Vault Assistant

The floating Vault Assistant provides quick prompts such as expo timings, how to bid, directions, what is live now and schedule a visit. It acts as an interface helper for common questions and navigation.

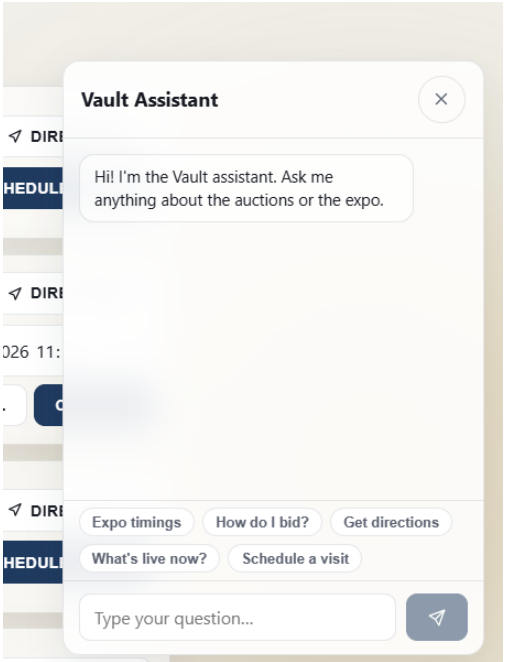


Figure 7.7 Vault Assistant with quick prompts.

7.8 Administrator Features — Separate Control Center

Administrator functions are deliberately segregated from the normal user experience. The control center exposes Overview, Approvals, Products, Auctions, Users and Bids. Admin access is enforced server-side through role checks rather than relying only on hiding UI links.

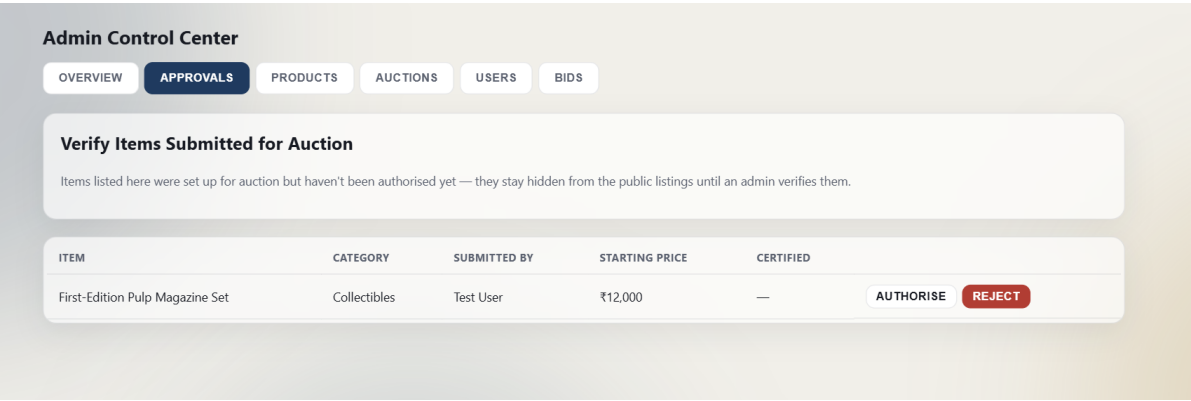


Figure 7.8a Administrator Approvals tab.

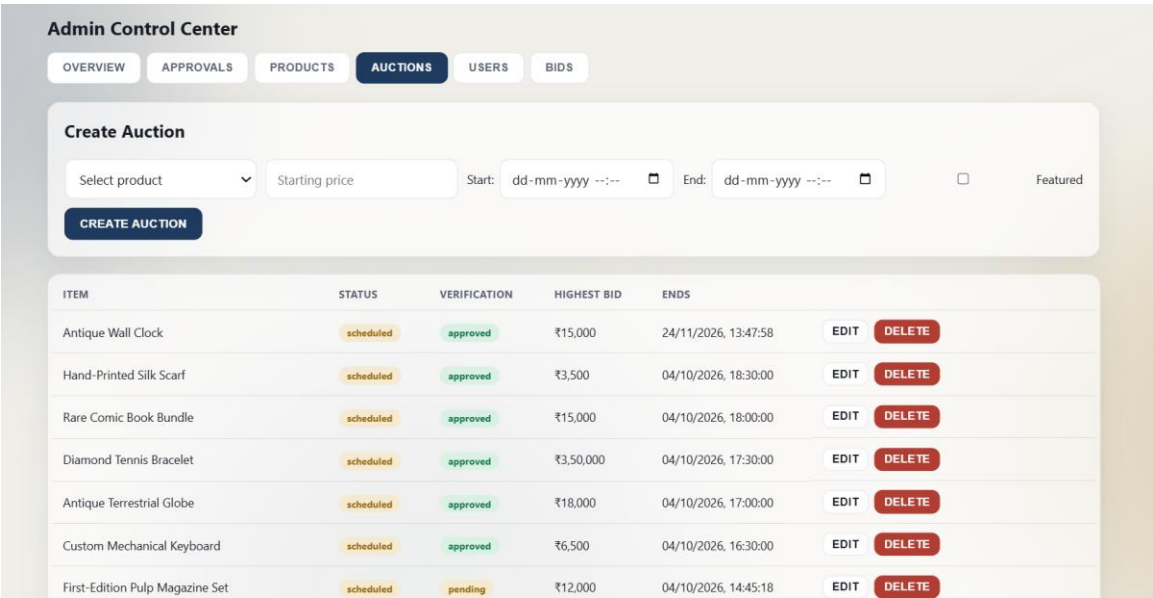


Figure 7.8b Administrator Auctions tab with status and verification.

CHAPTER -8 TESTING

8.1 Testing Strategy

Level	Technique	Focus
Unit / API	Manual REST calls (Postman / curl)	Validation messages and status codes in controllers
Authorization	Role-based request checks	JWT middleware and admin guard
Database	Schema constraints, SQL inspection	FK, composite PK, row state after bids
Concurrency	server/test/concurrentBidTest.js	No lost update under simultaneous bids
UI / real-time	Two browser sessions	bid_update and outbid events

Testing covers UI workflows, REST endpoints, database operations, authentication, authorization, auction lifecycle transitions and concurrent bidding. Because the core risk is inconsistent bid state, concurrency testing is treated separately from ordinary functional testing.

8.2 Authentication Test Cases

Registration with valid input should create a user and return an authentication response. Duplicate email should be rejected. Login with a valid password should succeed; an invalid password should fail. Protected endpoints should reject missing or invalid tokens.

ID	Test case	Input / condition	Expected result	Enforced in
A1	Register valid	name, email, password	201; token and user (role user); password stored as bcrypt hash	authController L17–29
A2	Register missing field	no password	400 name, email, and password are required	authController L8–10
A3	Duplicate email	existing email	409 Email already registered	authController L12–15
A4	Login valid	seeded user + password123	200; JWT valid 12 h with id, name, role	authController L43–63
A5	Login wrong password	bad password	401 Invalid credentials	authController L49–52
A6	No token	bid request without header	401 No token provided	auth.js L5–7
A7	Bad/expired token	tampered JWT	401 Invalid or expired token	auth.js L12–14

8.3 Authorization Test Cases

A normal authenticated user should not be able to execute administrator endpoints. An administrator should be able to access the control center operations. Invalid role values are rejected by the role-update endpoint.

ID	Test case	Input / condition	Expected result	Enforced in
Z1	User on admin route	user token, GET /api/admin/users	403 Admin access required	isAdmin.js
Z2	Admin on admin route	admin token, GET /api/admin/stats	200 with statistics JSON	adminRoutes.js
Z3	User creates auction	user token, POST /api/auctions	403 Admin access required	auctionRoutes.js
Z4	Invalid role value	PATCH role = "owner"	400 role must be user or admin	adminController L35–37

8.4 Auction and Product Test Cases

Product creation should persist the supplied fields. Auction creation should set active or scheduled status based on the start time. Editing an auction recalculates its lifecycle status from start and end times. Deletion should remove the target record when constraints permit.

ID	Test case	Input / condition	Expected result	Enforced in
P1	Create product	name present	201 with new id	productController L26–35
P2	Create product, no name	empty body	400 name is required	productController L29
U1	Create auction, past start	start_time ≤ now	status active	auctionController L48
U2	Create auction, future start	start_time > now	status scheduled	auctionController L48
U3	Edit auction times	new start/end	status recalculated	auctionController L74
U4	Delete auction with bids	FK RESTRICT	409 This auction has bids and cannot be deleted	auctionController L109–111
P3	Delete product with auctions	FK RESTRICT	409 This product has auctions and cannot be deleted	productController L65–67

8.5 Bid Validation Test Cases

A bid against a non-active auction should fail. A bid below or equal to the locked highest bid should fail. A bid after the end time should fail. A valid bid should update the auction and create exactly one bid history row within the same transaction.

ID	Test case	Input / condition	Expected result	Enforced in
B1	Auction not active	scheduled/closed auction	400 Auction is not active	bidController (status check)
B2	Bid after end time	now > end_time	400 Auction has already ended	bidController (end_time check)
B3	Bid $\leq$ current highest	equal or lower amount	409 Bid must be higher than current highest bid	bidController (amount check)
B4	Invalid amount	0 or missing	400 A valid bid amount is required	placeBid
B5	Unknown auction	nonexistent id	404 Auction not found	placeBidTransaction
B6	Valid bid	amount > current	201; auctions row updated and one bids row inserted in one commit	placeBidTransaction

8.6 Integration Testing

Integration testing verifies the complete path from browser action to REST endpoint, controller, database and returned JSON. Real-time integration additionally verifies that a successful bid results in the expected Socket.IO update and outbid notification.

Flow	Layers exercised	Expected observation
Login then place bid	React → Axios → route → authenticate → placeBid → MySQL	201 JSON; price updates on the page
Live update	Controller → io.emit(bid_update) → other browser	Second session shows new price without refresh
Outbid alert	io.to(user_id).emit(outbid)	Previous leader receives toast notification
Auction closure	node-cron → closeAuctionById → auction_results	Status closed; winner row created; auction_closed emitted
Admin approval	PATCH /api/admin/auctions/:id/approval	approval_status changes; public lists exclude pending/rejected

8.7 Concurrency Test

The repository includes `server/test/concurrentBidTest.js`. The test is designed to submit multiple bids against the same auction so that competing requests exercise the transaction and row lock. The expected invariant is that accepted bids remain strictly ordered and the final auction value matches the latest accepted bid.

```
• server/test/concurrentBidTest.js (lines 34-82)
34 async function main() {
35   const auctionId = process.argv[2] || 1;
36   const n = Math.min(Number(process.argv[3] || 4), TEST_USERS.length);
37
38   console.log(`\nLogging in ${n} test users...`);
39   const tokens = await Promise.all(
40     TEST_USERS.slice(0, n).map((u) => login(u.email, u.password))
41   );
42
43   const { data: before } = await axios.get(`${BASE_URL}/auctions/${auctionId}`);
44   console.log(`\nAuction ${auctionId} BEFORE test: current_highest_bid = ${before.current_highest_bid}`);
45
46   const baseBid = Number(before.current_highest_bid) + 100;
47   const amounts = tokens.map((_, i) => baseBid + i * 50); // distinct, increasing amounts
48   const expectedMax = Math.max(...amounts);
49
50   console.log(`\nFiring ${n} concurrent bids at auction ${auctionId}:`, amounts);
51
52   const results = await Promise.allSettled(
53     tokens.map((token, i) =>
54       axios.post(
55         `${BASE_URL}/auctions/${auctionId}/bids`,
56         { amount: amounts[i] },
57         { headers: { Authorization: `Bearer ${token}` } }
58       )
59     )
60   );
61
62   results.forEach((r, i) => {
63     if (r.status === 'fulfilled') {
64       console.log(` Bid ${amounts[i]} -> accepted (bidId=${r.value.data.bidId})`);
65     } else {
66       console.log(` Bid ${amounts[i]} -> rejected (${r.reason.response?.data?.error || r.reason.message})`);
67     }
68   });
69
70   const { data: after } = await axios.get(`${BASE_URL}/auctions/${auctionId}`);
71   const { data: history } = await axios.get(`${BASE_URL}/auctions/${auctionId}/bids`);
72
73   console.log(`\nAuction ${auctionId} AFTER test: current_highest_bid = ${after.current_highest_bid}`);
74   console.log(`Expected highest bid: ${expectedMax}`);
75   console.log(`Bids recorded in history table: ${history.length}`);
76
77   if (Number(after.current_highest_bid) === expectedMax) {
78     console.log(`\n✓PASS: current_highest_bid matches the true maximum of concurrent bids. No lost update.`);
79   } else {
80     console.log(`\n✗FAIL: current_highest_bid does NOT match expected maximum. Investigate transaction logic.`);
81   }
82 }
```

8.8 Database Integrity Testing

Foreign keys are used to prevent invalid references. Composite primary keys in watchlist and visits prevent duplicate user-auction relationships. The database therefore rejects several classes of inconsistent data before the application can expose them.

• server/schema.sql (lines 56-65)

```
56 CREATE TABLE bids (  
57   id INT AUTO_INCREMENT PRIMARY KEY,  
58   auction_id INT NOT NULL,  
59   user_id INT NOT NULL,  
60   amount DECIMAL(10,2) NOT NULL,  
61   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
62   FOREIGN KEY (auction_id) REFERENCES auctions(id) ON DELETE RESTRICT,  
63   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT,  
64   INDEX idx_auction_amount (auction_id, amount)  
65 ) ENGINE=InnoDB;
```

• server/schema.sql (lines 77-84)

```
77 CREATE TABLE watchlist (  
78   user_id INT NOT NULL,  
79   auction_id INT NOT NULL,  
80   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
81   PRIMARY KEY (user_id, auction_id),  
82   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
83   FOREIGN KEY (auction_id) REFERENCES auctions(id) ON DELETE CASCADE  
84 ) ENGINE=InnoDB;
```

8.9 Test Summary

The implementation can be evaluated across functional correctness, authorization, transaction correctness, real-time behaviour and database integrity. The concurrency test is especially relevant to the DSDBE learning objectives because it demonstrates the difference between an ordinary CRUD operation and a transaction that must preserve an invariant under competing requests.

Area	Method	Evidence
Authentication / authorization	Test cases A1–A7, Z1–Z4	Controller and middleware responses
Auction and product CRUD	Test cases P1–P3, U1–U4	Status codes, FK errors
Bid validation	Test cases B1–B6	Transactional checks under row lock
Concurrency	npm run test:concurrency	Final highest bid equals maximum submitted
Integrity	Schema constraints	FK and composite PK reject invalid rows

CHAPTER -9 DEPLOYMENT

9.1 Local Project Setup

```
• terminal - project root
1 # 1. database
2 mysql -u root -p < server/schema.sql
3 mysql -u root -p < server/seed.sql
4
5 # 2. backend
6 cd server && cp .env.example .env && npm install && npm run dev
7
8 # 3. frontend
9 cd client && cp .env.example .env && npm install && npm run dev
```

The repository contains client and server directories. The server must be connected to MySQL and configured through its environment file. The client is configured with the backend/API origin. Both sides install their own package dependencies.

9.2 Database Initialization

The database schema is provided in `server/schema.sql` and demonstration records are provided in `server/seed.sql`. The schema should be applied before the server is started so that controllers can access the required tables.

```
• terminal
1 mysql -u root -p < server/schema.sql
2 mysql -u root -p < server/seed.sql
```

```
• server/schema.sql (lines 1-12)
1 -- Online Auction & Bidding Platform - Schema
2 -- Engine MUST be InnoDB (row-level locking / transactions require it)
3
4 CREATE DATABASE IF NOT EXISTS auction_platform;
5 USE auction_platform;
6
7 DROP TABLE IF EXISTS visits;
8 DROP TABLE IF EXISTS watchlist;
9 DROP TABLE IF EXISTS auction_results;
10 DROP TABLE IF EXISTS bids;
11 DROP TABLE IF EXISTS auctions;
12 DROP TABLE IF EXISTS products;
```

9.3 Server Environment

The server environment contains database connection details, JWT secret and client origin. These values should be supplied for the local environment rather than hard-coded into controller files.

```
• server/.env.example
1 PORT=5000
2 DB_HOST=localhost
3 DB_PORT=3306
4 DB_USER=root
5 DB_PASSWORD=yourpassword
6 DB_NAME=auction_platform
7 JWT_SECRET=change_this_to_a_long_random_secret
8 CLIENT_ORIGIN=http://localhost:5173
```

9.4 Client Environment

The client environment contains the backend API and socket connection settings. Vite exposes client-side environment variables using its supported naming convention.

```
• client/.env.example
1 VITE_API_URL=http://localhost:5000/api
2 VITE_SOCKET_URL=http://localhost:5000
```

9.5 Starting the Backend

From the server directory, dependencies are installed with `npm install` and the development command is used to start the Node server. The backend exposes REST routes and the Socket.IO server from the same application process.

```
• terminal - server/  
1 cd server  
2 cp .env.example .env # set DB_PASSWORD and JWT_SECRET  
3 npm install  
4 npm run dev # nodemon server.js, port 5000
```

9.6 Starting the Frontend

From the client directory, dependencies are installed and Vite is started. The browser can then open the local Vite address and communicate with the backend.

```
• terminal - client/  
1 cd client  
2 cp .env.example .env  
3 npm install  
4 npm run dev          # Vite on http://localhost:5173
```

9.7 Concurrency Demonstration

The README documents the concurrency demonstration command. It is run while the backend and database are active so that multiple requests can be generated against the same auction.

```
• terminal - server/  
1 npm run test:concurrency -- 1 4  
2 # args: <auctionId> <numConcurrentBids>
```

```
• MySQL client  
1 SELECT current_highest_bid, current_highest_bidder_id FROM auctions WHERE id = 1;  
2 SELECT * FROM bids WHERE auction_id = 1 ORDER BY created_at;
```

9.8 Deployment Considerations

For a hosted deployment, the same client/server/database separation can be retained. Environment variables should be supplied through the hosting platform, the database should use secure credentials, and HTTPS should protect browser-to-server communication.

Concern	Recommendation
Secrets	Supply DB credentials and JWT_SECRET through host environment variables
Transport	Serve client and API over HTTPS
Database	Dedicated MySQL user with least privilege; scheduled backups
CORS	Set CLIENT_ORIGIN to the deployed frontend URL

9.9 Deployment Summary

The deployment process is reproducible because the repository contains package manifests, SQL schema, seed data, environment examples and a README. The local setup is therefore suitable for demonstration, testing and academic evaluation.

Artifact	Purpose
package.json / lock files	Reproducible dependencies
schema.sql, seed.sql	Database creation and demo data
.env.example files	Required configuration keys
README.md	Run and concurrency-demo steps

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Concurrent Bid Correctness

```
• server/controllers/bidController.js (lines 15-46)
15 async function placeBidTransaction(auctionId, userId, amount) {
16   const connection = await pool.getConnection();
17   try {
18     await connection.beginTransaction();
19
20     // Row lock: any other transaction trying to SELECT ... FOR UPDATE the
21     // same auction row will block here until this transaction commits or
22     // rolls back.
23     const [rows] = await connection.query(
24       'SELECT current_highest_bid, current_highest_bidder_id, status, end_time FROM auctions WHERE id = ? FOR
UPDATE',
25       [auctionId]
26     );
27
28     if (rows.length === 0) {
29       await connection.rollback();
30       return { success: false, status: 404, error: 'Auction not found' };
31     }
32
33     const auction = rows[0];
34
35     if (auction.status !== 'active') {
36       await connection.rollback();
37       return { success: false, status: 400, error: 'Auction is not active' };
38     }
39
40     if (new Date() > new Date(auction.end_time)) {
41       await connection.rollback();
42       return { success: false, status: 400, error: 'Auction has already ended' };
43     }
44
45     if (Number(amount) <= Number(auction.current_highest_bid)) {
46       await connection.rollback();
```

The most significant challenge is concurrent bidding. A UI-only validation cannot guarantee correctness because another request may commit a newer bid between the client's read and write. The solution is to move the critical validation into a database transaction and lock the auction row.

10.2 Deadlocks and Transaction Failure

Row locking introduces the possibility of deadlock in more complex concurrent workloads. The implementation therefore includes retry handling for transient deadlock conditions. Rollback also ensures partial work is not retained when the transaction fails.

```
• server/controllers/bidController.js (lines 87-99)
87 async function placeBidWithRetry(auctionId, userId, amount, retries = 3, attempt = 1) {
88   try {
89     return await placeBidTransaction(auctionId, userId, amount);
90   } catch (err) {
91     const isDeadlock = err.code === 'ER_LOCK_DEADLOCK' || err.errno === 1213;
92     if (isDeadlock && retries > 0) {
93       const backoff = 50 * attempt + Math.random() * 100;
94       await sleep(backoff);
95       return placeBidWithRetry(auctionId, userId, amount, retries - 1, attempt + 1);
96     }
97     throw err;
98   }
99 }
```

10.3 Keeping UI State Current

A second challenge is presenting the latest bid without excessive polling. Socket.IO solves this by broadcasting committed updates to auction rooms and sending targeted outbid notifications.

```
• client/src/pages/AuctionDetail.jsx (lines 61-75)
61   function handleBidUpdate(update) {
62     if (Number(update.auctionId) !== Number(id)) return;
63     setAuction((prev) => prev && { ...prev, current_highest_bid: update.currentHighestBid });
64     if (update.bidderId !== user?.id) toast(`New bid: ₹${update.currentHighestBid} by ${update.bidderName}`, {
65       icon: <IconBell width={16} height={16} /> });
66     refresh(); // re-pull bid history for the table
67   }
68   function handleAuctionClosed(info) {
69     if (Number(info.auctionId) !== Number(id)) return;
70     setClosedInfo(info);
71     setAuction((prev) => prev && { ...prev, status: 'closed' });
72   }
73
74   socket.on('bid_update', handleBidUpdate);
75   socket.on('auction_closed', handleAuctionClosed);
```

10.4 Auction Lifecycle Automation

Auctions have three meaningful states in the implementation: scheduled, active and closed. The scheduled job prevents lifecycle state from depending entirely on manual administrator actions.

```
• server/jobs/closeAuctions.js (lines 80-92)
80 async function activateStartedAuctions(io) {
81   const [due] = await pool.query(
82     `SELECT id FROM auctions
83      WHERE status = 'scheduled' AND start_time <= NOW() AND end_time > NOW()`
84   );
85   for (const auction of due) {
86     const [result] = await pool.query(
87       `UPDATE auctions SET status = 'active' WHERE id = ? AND status = 'scheduled'`,
88       [auction.id]
89     );
90     if (result.affectedRows > 0 && io) io.emit('auction_started', { auctionId: auction.id });
91   }
92 }
```

10.5 Role Separation

Because the application contains both normal-user and administrator functionality, authorization must be enforced on the server. Hiding Admin navigation items in React would not be sufficient protection by itself.

```
• server/routes/adminRoutes.js
1  const express = require('express');
2  const router = express.Router();
3  const authenticate = require('../middleware/auth');
4  const isAdmin = require('../middleware/isAdmin');
5  const { listUsers, listAllBids, updateUserRole, deleteUser, getStats, listPendingAuctions, setAuctionApproval } =
      require('../controllers/adminController');
6
7  router.get('/users', authenticate, isAdmin, listUsers);
8  router.patch('/users/:id/role', authenticate, isAdmin, updateUserRole);
9  router.delete('/users/:id', authenticate, isAdmin, deleteUser);
10 router.get('/bids', authenticate, isAdmin, listAllBids);
11 router.get('/stats', authenticate, isAdmin, getStats);
12 router.get('/auctions/pending', authenticate, isAdmin, listPendingAuctions);
13 router.patch('/auctions/:id/approval', authenticate, isAdmin, setAuctionApproval);
14
15 module.exports = router;
```

10.6 External Location Services

Directions depend on browser location permission and external map/geocoding/routing services. Network availability or permission settings can therefore affect the live-auction directions experience without changing the underlying auction data.

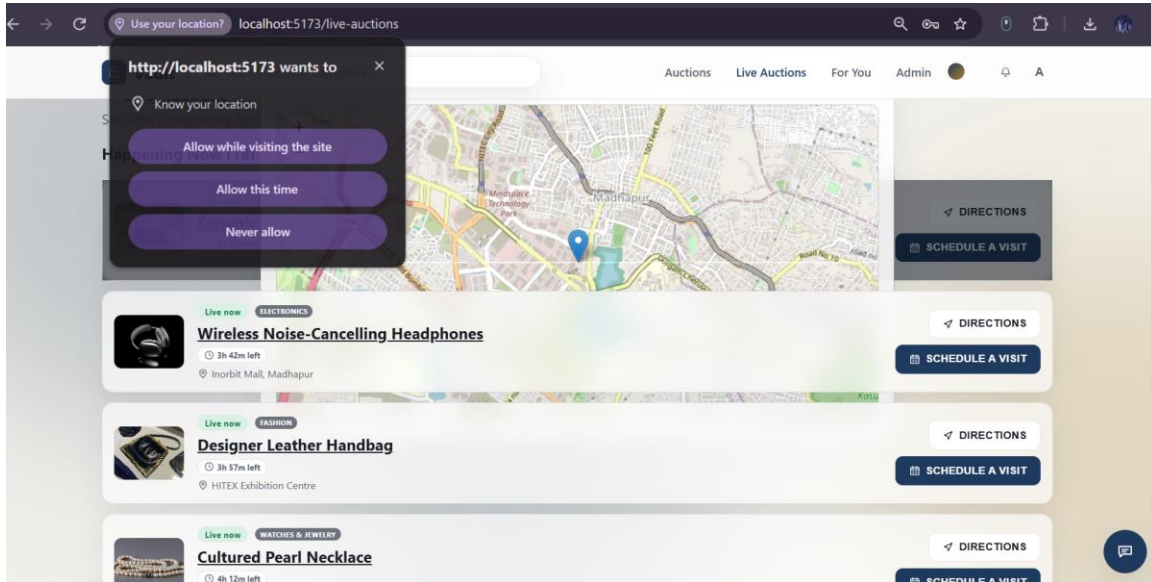


Figure 10.6 Location permission prompt over the Live Auctions map.

10.7 Current Limitations

The current build is a single MySQL deployment. It does not implement read replicas, horizontal sharding or distributed consensus. Advanced query-plan analysis with a dedicated EXPLAIN workflow is also not a primary implemented feature.

Limitation	Current state	Future remedy
Single database node	One MySQL instance	Primary-replica replication
No sharding	All rows on one server	Shard bids by auction_id
No EXPLAIN workflow	Index added by design only	Systematic EXPLAIN ANALYZE tuning

10.8 Data and Operational Limitations

The project uses image URLs rather than a full image-storage pipeline. Notification persistence and richer recommendation models are possible future extensions. These limitations do not change the core transaction and auction workflow.

Area	Limitation	Impact
Images	Stored as URLs only	No upload pipeline
Notifications	Held in client state	Lost on reload
Recommendations	Category-based rules	No learning model

10.9 Lessons from the Challenges

The project shows that a database-backed application should place correctness-critical rules close to the data. It also demonstrates that real-time UI behaviour, scheduled jobs and authorization should complement—not replace—database constraints and transaction boundaries.

Lesson	Consequence
Keep invariants near the data	Row lock + transaction decides bid validity
UI is not a security boundary	Role checks enforced server-side
Real-time complements the DB	Socket events follow committed state

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Payment Integration

Planned item	Description
payments table	payment_id, auction_id, payer_id, amount, status
Link	Reference auction_results.auction_id
Separation	Payments never modify bids history

A future version can add secure payment workflows after an auction closes, with payment status linked to auction_results. Payment state should be modelled separately from the bid history so that financial processing does not alter the historical bidding record.

11.2 Database and Scalability Enhancements

Read replicas can support high-volume read traffic. Sharding can be considered for very large bid and auction datasets. Caching can reduce repeated public listing reads. EXPLAIN and EXPLAIN ANALYZE workflows can be added for systematic query tuning.

Technique	Benefit
Read replicas	Scale listing and history reads
Sharding by auction_id	Spread bid volume
Caching	Reduce repeated public reads
EXPLAIN ANALYZE	Evidence-based tuning

11.3 Operational Observability

Future deployments can record transaction latency, lock waits, deadlocks and retry counts. Automated backups and tested restore procedures can improve operational resilience.

Metric	Why it matters
Transaction latency	Bid response time
Lock waits and deadlocks	Contention on hot auctions
Retry count	Frequency of ER_LOCK_DEADLOCK retries

11.4 User Experience Enhancements

The interface can be refined for mobile-first use. Auction analytics, price history charts, calendar integration for visits, seller dashboards and richer assistant content can extend the user experience.

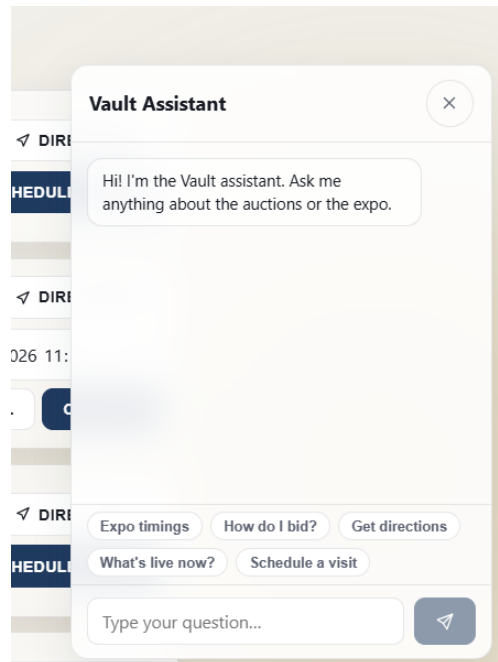


Figure 11.4 Vault Assistant, a candidate for richer help content.

11.5 Distributed Database Scope

The DSDBE handout identifies replication, sharding and CAP as distributed-database fundamentals. The current implementation remains single-node, so these topics are appropriate future scalability work rather than claims about the existing deployment.

CO6 concept	Relevance to Vault
Primary-replica replication	Offload read-heavy listings
Horizontal sharding	Partition bids by auction_id
CAP theorem	Bidding favours consistency; reads may tolerate lag

11.6 Future Scope Summary

Future work should preserve the existing auction invariant while scaling the read path and adding operational services around it. The transaction boundary for bid acceptance remains the foundation on which these extensions should be built.

Phase	Focus
1	Payments after auction close
2	Observability and backups
3	Replicas and caching
4	Sharding and analytics

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

Area	Outcome
Database	7-table InnoDB schema with FK and composite keys
Concurrency	Transactional bids with SELECT ... FOR UPDATE
Real-time	Socket.IO bid and outbid events
Automation	node-cron activates and closes auctions
Admin	Separate protected control center

Vault delivers a complete web-based auction workflow using React, Node.js, Express, MySQL and Socket.IO. It combines user-facing auction discovery with a separately protected administrator control center.

12.2 Database Design Achievement

The relational schema separates users, products, auctions, bids, auction results, watchlist entries and visits. Keys and foreign keys maintain integrity, while composite keys prevent duplicate user-auction relationships.

Concern	Design choice
Referential integrity	Foreign keys with RESTRICT / CASCADE / SET NULL
Duplicates	Composite PK on watchlist and visits
Retrieval	idx_auction_amount (auction_id, amount)

12.3 Concurrency Achievement

The central technical achievement is transaction-based bid processing with `SELECT ... FOR UPDATE`. The design ensures that competing requests do not validate against a stale auction value and then overwrite the current state.

```
• server/controllers/bidController.js (lines 18-31)
18   await connection.beginTransaction();
19
20   // Row lock: any other transaction trying to SELECT ... FOR UPDATE the
21   // same auction row will block here until this transaction commits or
22   // rolls back.
23   const [rows] = await connection.query(
24     'SELECT current_highest_bid, current_highest_bidder_id, status, end_time FROM auctions WHERE id = ? FOR
UPDATE',
25     [auctionId]
26   );
27
28   if (rows.length === 0) {
29     await connection.rollback();
30     return { success: false, status: 404, error: 'Auction not found' };
31   }
```

12.4 Real-Time Achievement

Socket.IO extends the transaction result to connected clients. A committed bid can trigger a bid-update broadcast and an outbid notification without requiring every browser to poll the database continuously.

```
• client/src/context/NotificationContext.jsx (lines 17-27)
17   function handleOutbid(payload) {
18     const entry = {
19       id: `${payload.auctionId}-${Date.now()}`,
20       auctionId: payload.auctionId,
21       message: `You've been outbid on "${payload.productName}" - new bid ₹${payload.newAmount} by
    ${payload.outbidBy}.`,
22       time: new Date().toISOString(),
23       read: false,
24     };
25     setNotifications((prev) => [entry, ...prev].slice(0, 20));
26     toast(entry.message, { icon: <IconBell width={16} height={16} /> });
27   }
```

12.5 Administrator Achievement

The administrator control center separates operational management from normal bidding. Approvals, products, auctions, users and bids have dedicated interfaces and protected backend routes.

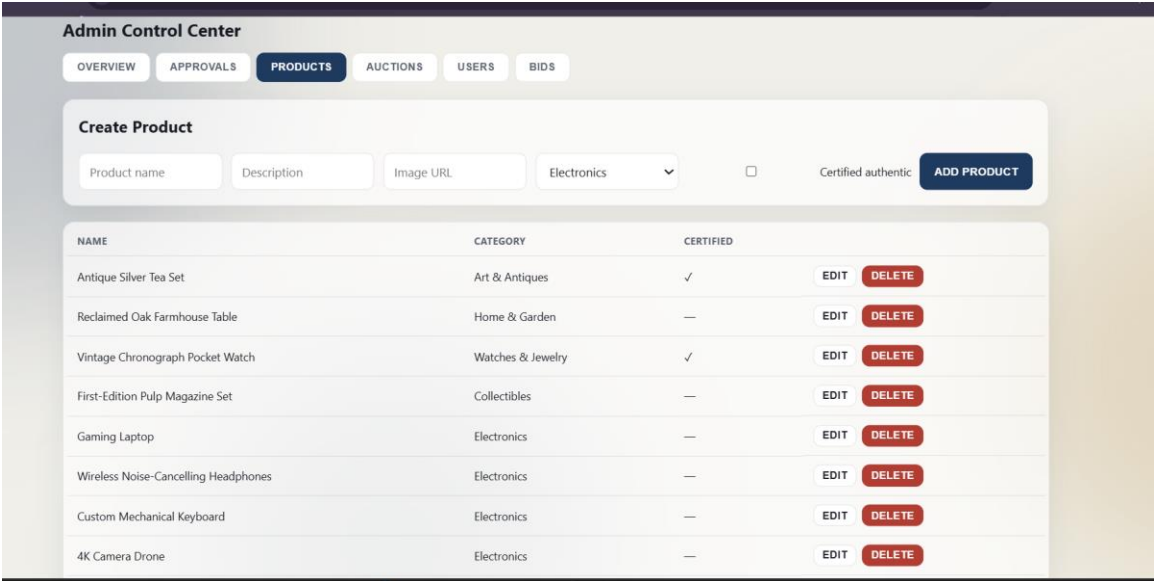


Figure 12.5 Administrator product management.

12.6 User Experience Achievement

The user interface supports discovery, filters, bidding, watchlists, recommendations, activity, live venues, visit scheduling, directions and an assistant. These features provide the application layer around the database core.

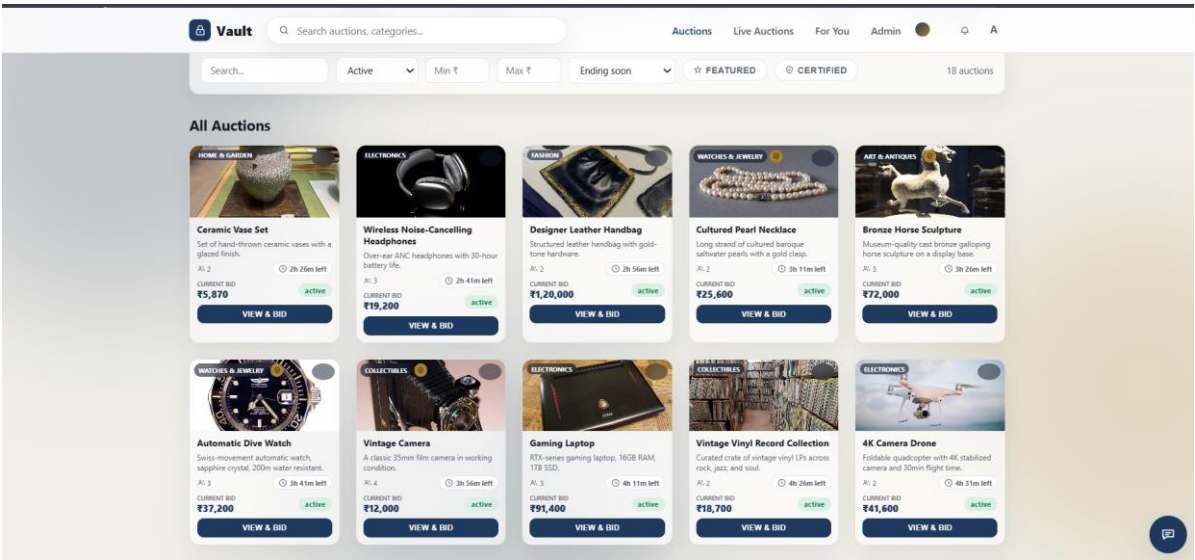


Figure 12.6 User-facing auction browsing.

12.7 Academic Learning

The project provided practical experience in relational modelling, SQL, backend layering, JWT authentication, transactions, row-level locking, concurrency testing, indexing and real-time communication. It also provides a concrete case for discussing distributed database techniques as future scope.

Course outcome	Applied in Vault
CO1	Routes → middleware → controllers → MySQL
CO2	ER design, keys, constraints
CO3	SQL joins, GROUP BY, filtering
CO4	Transactions, row locks, deadlock retry
CO5	Composite index on bids

12.8 Overall Project Outcome

The implementation connects database theory to a realistic workload in which correctness matters. The auction domain makes transaction boundaries, locking and invariants observable and testable.

Objective	Status
Concurrency-safe bidding	Implemented
Bid history and results	Implemented
Real-time updates	Implemented
Replication / sharding	Future scope

12.9 Final Conclusion

The Online Auction & Bidding Platform demonstrates how a full-stack application can be designed around a relational database while preserving correctness under concurrent activity. The project meets its core implementation scope and provides a clear foundation for later scalability, payment and analytics enhancements.

Aspect	Summary
Core strength	Correct bid state under concurrent requests
Foundation for	Payments, replicas and analytics

CHAPTER -13 REFERENCES

13.1 Technical References

Resource	Used for
React, Vite	Frontend
Express, Node.js	Backend
MySQL 8.0 / InnoDB	Transactions and locking
Socket.IO	Real-time events
Leaflet / OpenStreetMap	Maps and routing

React Documentation — component-based frontend architecture.

Vite Documentation — frontend development and build tooling.

Express.js Documentation — REST routing and middleware.

MySQL 8.0 Documentation — SQL, InnoDB, transactions and locking.

Node.js Documentation — server-side JavaScript runtime.

Socket.IO Documentation — real-time bidirectional communication.

Leaflet Documentation — interactive map rendering.

OpenStreetMap / Nominatim / OSRM documentation — map, geocoding and routing services.

Project README and source repository — implemented architecture, setup and concurrency workflow.

DSDBE Revised 2026–27 Course Handout — course outcomes, CO mapping, PO/PSO mapping and delivery plan.

13.2 Source Files Referenced in the Documentation

The implementation discussion in this report is based on the submitted project source. The principal files referenced include server/controllers/authController.js, auctionController.js, bidController.js, productController.js, adminController.js, recommendationController.js, userController.js, visitController.js and watchlistController.js. Additional implementation evidence comes from server/middleware/auth.js, server/middleware/isAdmin.js, server/jobs/closeAuctions.js, server/sockets/index.js, server/schema.sql, server/test/concurrentBidTest.js, route files, client/src/App.jsx, client/src/main.jsx and client/src/styles.css.

File	Role
controllers/authController.js	Register and login
controllers/bidController.js	Bid transaction and retry
controllers/auctionController.js	Auction CRUD and results
jobs/closeAuctions.js	Lifecycle automation
middleware/auth.js, isAdmin.js	JWT and role guard
test/concurrentBidTest.js	Concurrency proof

CHAPTER -14 APPENDICES

Appendix A — Course Outcome / PO / PSO Mapping

CO	Course outcome focus	Project evidence	PO/PSO	Evidence
CO1	Layered backend and request-response flow	Routes, auth/admin middleware, controllers, MySQL pool, REST endpoints	PO1, PO2, PO3	Direct evidence
CO2	ER modelling and 1NF/2NF/3NF/BCNF introduction	Seven-table relational model, PK/FK relationships, composite keys	PO1, PO3, PSO1	Direct evidence
CO3	SQL DDL/DML, joins, aggregation, subqueries/CTEs/window functions	schema.sql and controller queries with joins/grouping/filtering	PO1, PO3, PSO1	Strong; advanced topics not primary
CO4	Transactions, isolation, concurrency anomalies, MVCC intuition	InnoDB transaction, SELECT ... FOR UPDATE, commit/rollback, deadlock retry and concurrency test	PO1, PO2	Direct evidence
CO5	Indexes and query execution	Auction/amount composite index and relational query patterns	PO1, PO2	Partial; no dedicated EXPLAIN workflow
CO6	Replication, sharding and CAP	Single MySQL deployment; distributed scaling discussed as future scope	PO1, PO3, PO4	Conceptual/future scope

The mapping follows the DSDBE Revised 2026–27 handout. CO1 is associated with PO1/PO2/PO3; CO2 and CO3 with PO1/PO3/PSO1; CO4 and CO5 with PO1/PO2; and CO6 with PO1/PO3/PO4. The project provides direct implementation evidence for the layered backend, relational model, SQL, transaction and indexing portions, while distributed replication/sharding remains future scope.

Appendix B — Application Screenshots

Public Auction Interface

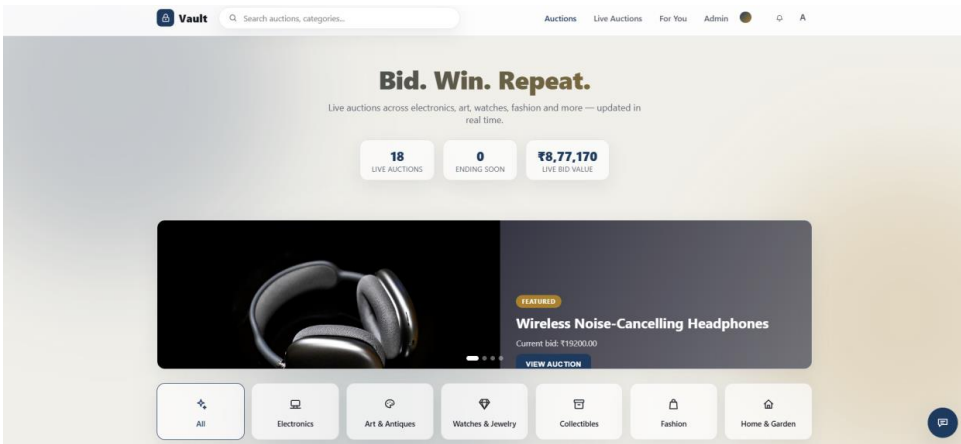


Figure B.1 Public auction home with live auction summary and featured item.

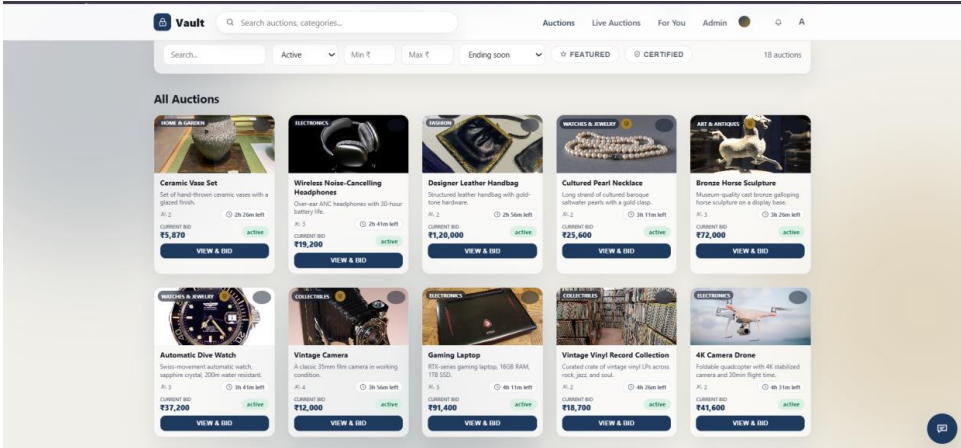


Figure B.2 Auction listing with search, status, price and category filters.

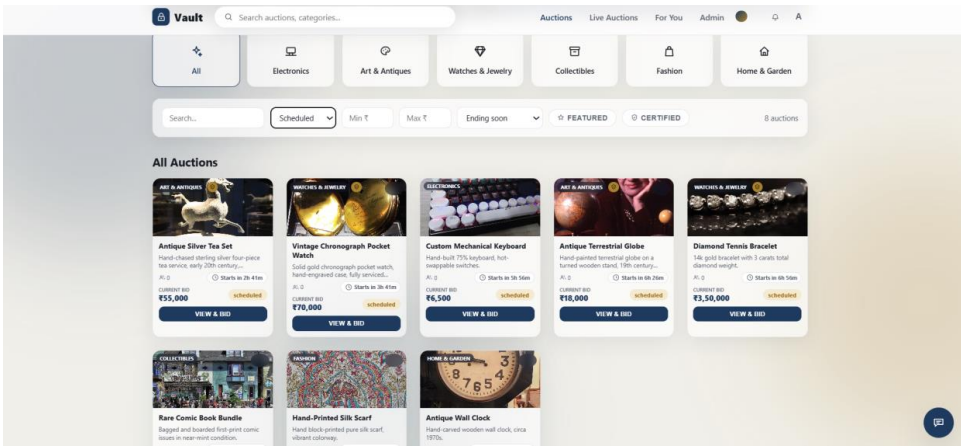


Figure B.3 Scheduled auction listing with future start times.

Appendix B — Application Screenshots

Authentication and Approval Workflow

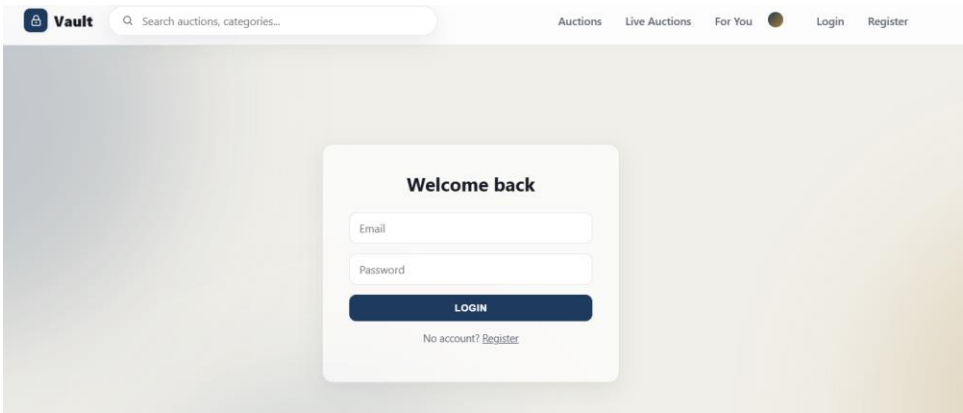


Figure B.4 User login page.

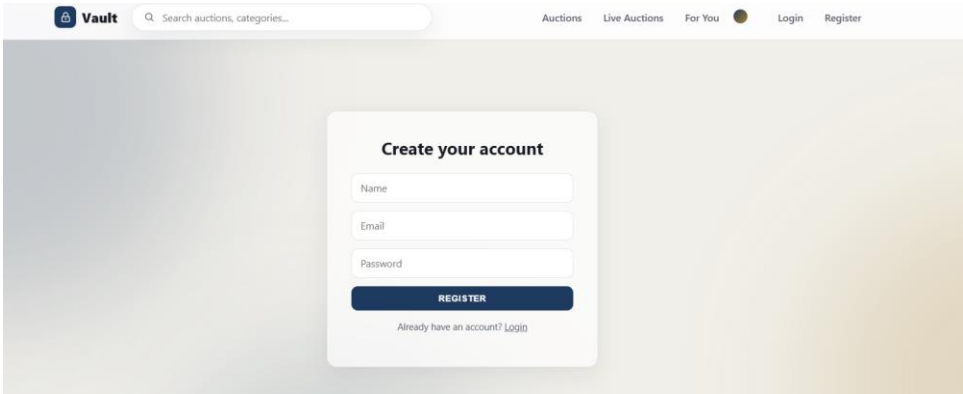


Figure B.5 User registration page.

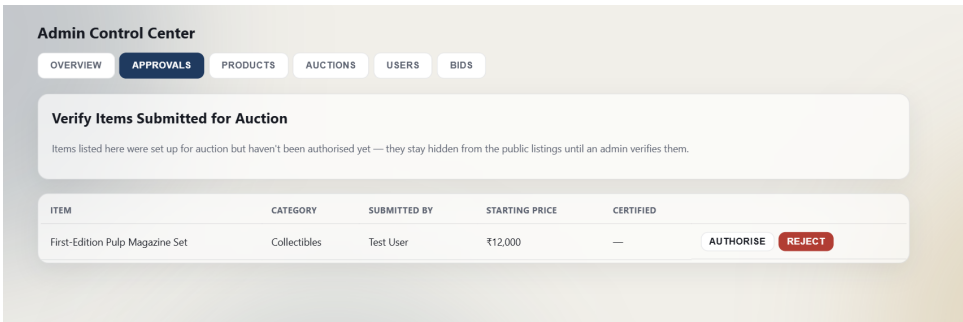


Figure B.6 Administrator approval screen for submitted auction items.

Appendix C — Administrator Screens

Product, Auction and Bid Administration

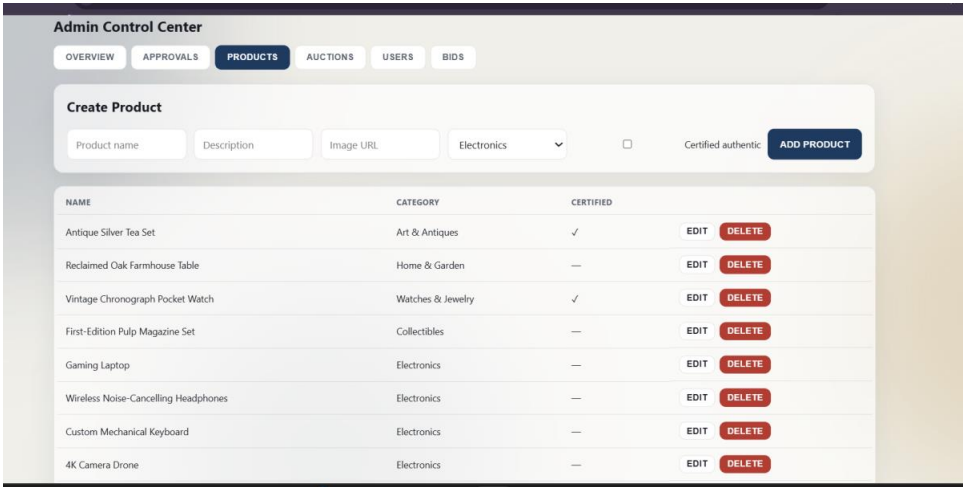


Figure C.1 Administrator product management.

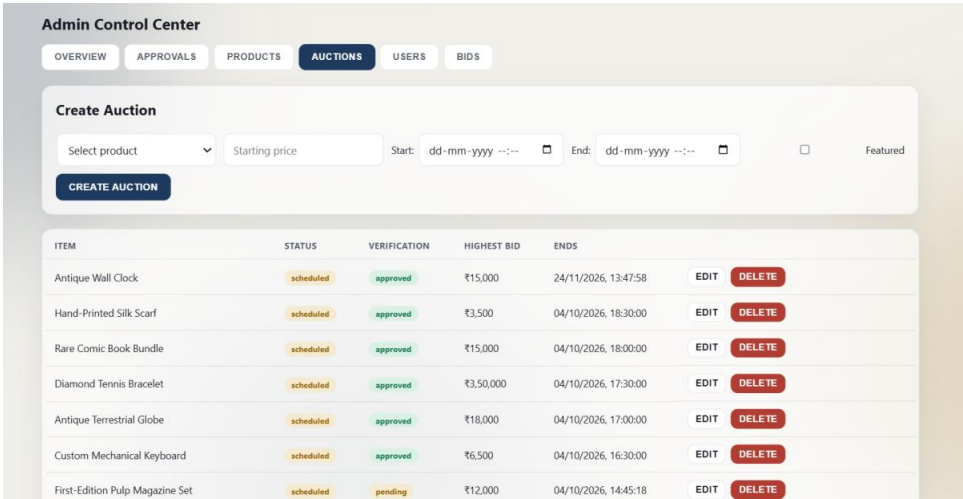


Figure C.2 Administrator auction management.

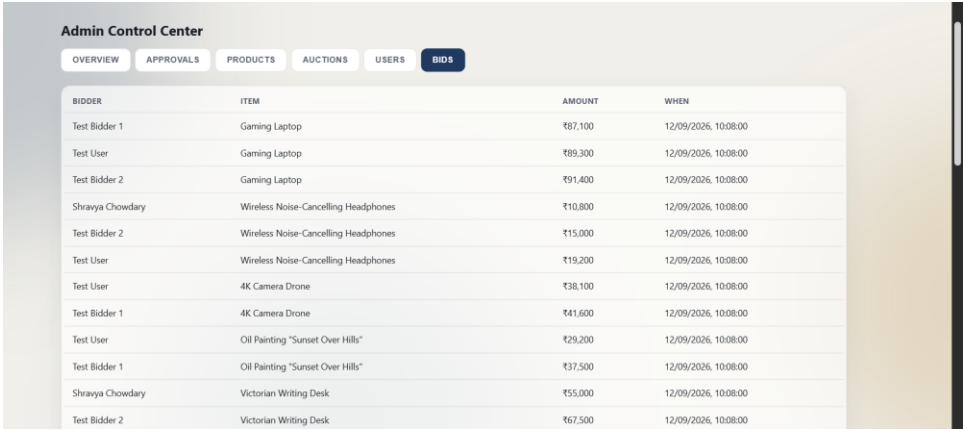


Figure C.3 Administrator bid records.

Appendix C — Live Auction Screens

Venue, Directions and Visit Scheduling

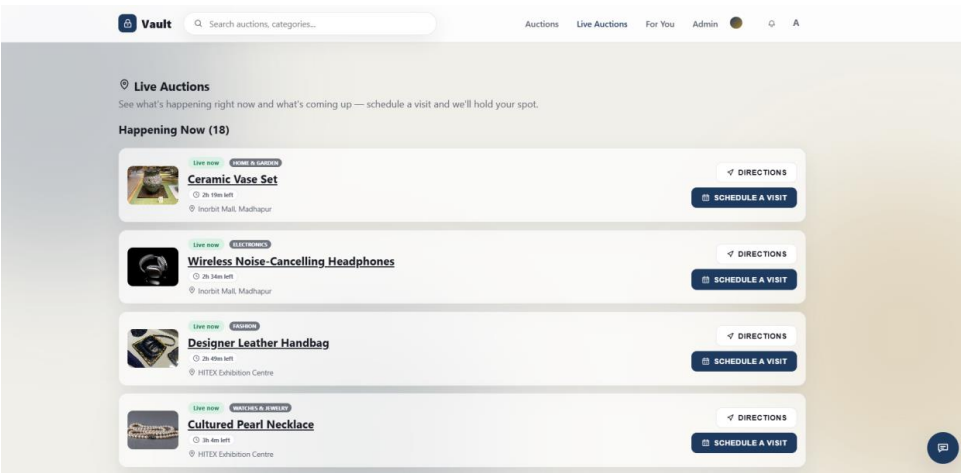


Figure C.4 Live Auctions page with venue information.

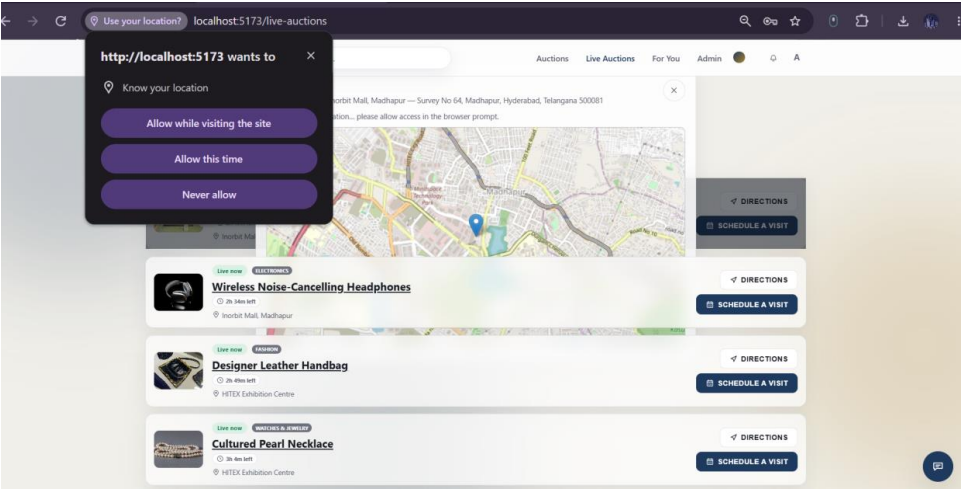


Figure C.5 Browser location permission and map workflow.

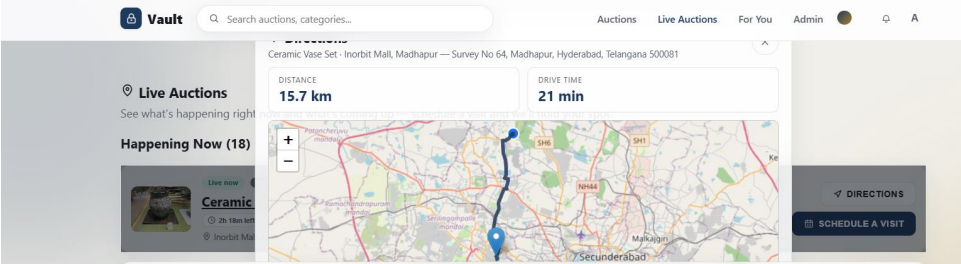


Figure C.6 Route/distance view and visit scheduling workflow.

Appendix D — Code, Setup and User Guide

Assistant, Additional UI Evidence and Source Snippets

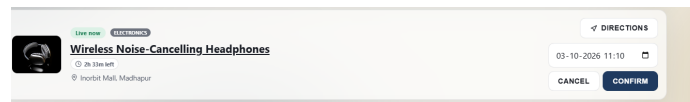


Figure D.1 Visit confirmation interface.

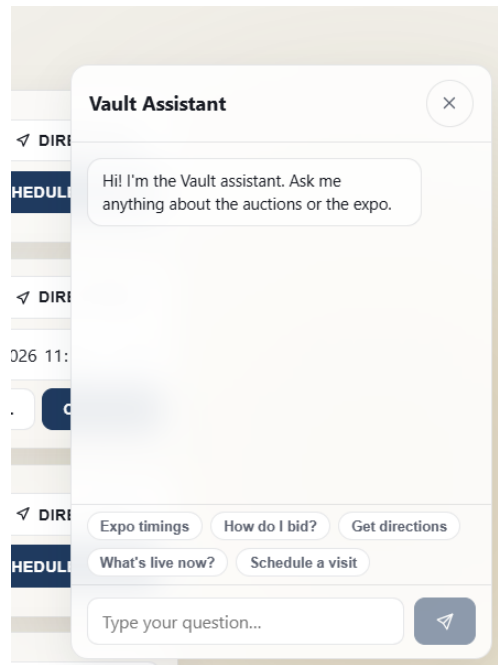


Figure D.2 Vault Assistant interface.

Selected implementation evidence:

```
• server/schema.sql
1  -- Online Auction & Bidding Platform - Schema
2  -- Engine MUST be InnoDB (row-level locking / transactions require it)
3
4  CREATE DATABASE IF NOT EXISTS auction_platform;
5  USE auction_platform;
6
7  DROP TABLE IF EXISTS visits;
8  DROP TABLE IF EXISTS watchlist;
9  DROP TABLE IF EXISTS auction_results;
10 DROP TABLE IF EXISTS bids;
11 DROP TABLE IF EXISTS auctions;
12 DROP TABLE IF EXISTS products;
13 DROP TABLE IF EXISTS users;
14
15 CREATE TABLE users (
16   id INT AUTO_INCREMENT PRIMARY KEY,
17   name VARCHAR(100) NOT NULL,
18   email VARCHAR(150) NOT NULL UNIQUE,
19   password_hash VARCHAR(255) NOT NULL,
20   role ENUM('user','admin') NOT NULL DEFAULT 'user',
21   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
22 ) ENGINE=InnoDB;
23
24 CREATE TABLE products (
25   id INT AUTO_INCREMENT PRIMARY KEY,
26   name VARCHAR(150) NOT NULL,
27   description TEXT,
28   image_url VARCHAR(500),
29   category VARCHAR(50),
```

```

30  is_certified BOOLEAN NOT NULL DEFAULT FALSE,
31  created_by INT,
32  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
33  FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
34 ) ENGINE=InnoDB;
35
36 CREATE TABLE auctions (
37   id INT AUTO_INCREMENT PRIMARY KEY,
38   product_id INT NOT NULL,
39   starting_price DECIMAL(10,2) NOT NULL,
40   current_highest_bid DECIMAL(10,2) NOT NULL DEFAULT 0,
41   current_highest_bidder_id INT NULL,
42   start_time DATETIME NOT NULL,
43   end_time DATETIME NOT NULL,
44   status ENUM('scheduled','active','closed') NOT NULL DEFAULT 'scheduled',
45   approval_status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'approved',
46   is_featured BOOLEAN NOT NULL DEFAULT FALSE,
47   venue_name VARCHAR(150) NULL,
48   venue_address VARCHAR(300) NULL,
49   venue_lat DECIMAL(9,6) NULL,
50   venue_lng DECIMAL(9,6) NULL,
51   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
52   FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE RESTRICT,
53   FOREIGN KEY (current_highest_bidder_id) REFERENCES users(id) ON DELETE SET NULL
54 ) ENGINE=InnoDB;
55
56 CREATE TABLE bids (
57   id INT AUTO_INCREMENT PRIMARY KEY,
58   auction_id INT NOT NULL,
59   user_id INT NOT NULL,
60   amount DECIMAL(10,2) NOT NULL,
61   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
62   FOREIGN KEY (auction_id) REFERENCES auctions(id) ON DELETE RESTRICT,
63   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT,
64   INDEX idx_auction_amount (auction_id, amount)
65 ) ENGINE=InnoDB;
66
67 CREATE TABLE auction_results (
68   auction_id INT PRIMARY KEY,
69   winning_bid_id INT,
70   winner_id INT,
71   finalized_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
72   FOREIGN KEY (auction_id) REFERENCES auctions(id) ON DELETE RESTRICT,
73   FOREIGN KEY (winning_bid_id) REFERENCES bids(id) ON DELETE SET NULL,
74   FOREIGN KEY (winner_id) REFERENCES users(id) ON DELETE SET NULL
75 ) ENGINE=InnoDB;
76
77 CREATE TABLE watchlist (
78   user_id INT NOT NULL,
79   auction_id INT NOT NULL,
80   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
81   PRIMARY KEY (user_id, auction_id),
82   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
83   FOREIGN KEY (auction_id) REFERENCES auctions(id) ON DELETE CASCADE
84 ) ENGINE=InnoDB;
85
86 CREATE TABLE visits (
87   user_id INT NOT NULL,
88   auction_id INT NOT NULL,
89   visit_time DATETIME NOT NULL,
90   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
91   PRIMARY KEY (user_id, auction_id),
92   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
93   FOREIGN KEY (auction_id) REFERENCES auctions(id) ON DELETE CASCADE
94 ) ENGINE=InnoDB;

```

Normal user guide: register/login → browse auctions → filter/search → open auction → review bid history → place bid → monitor live updates/outbid notifications → use watchlist/activity → use Live Auctions for directions and visit scheduling.

Administrator guide: login with admin role → Overview → Approvals → Products → Auctions → Users → Bids. Administrative operations are protected server-side by the role middleware.

Setup: apply server/schema.sql → load server/seed.sql → configure environment files → npm install in client and server → start backend → start Vite frontend → run the documented concurrency test when required.

